DEPARTMENT OF INFORMATICS
Mendel University in Brno

Bachelor's Thesis

Classification of Turbofan Engine Degradation Using Machine Learning Methods on C-MAPSS Data

Author: Artsiom Halachkin
Supervisor: doc. Ing. Oldřich Trenz, Ph.D

Brno 2026

Acknowledgements

I would like to express my gratitude to my thesis supervisor doc. Ing. Oldřich Trenz, Ph.D for continued support and guidance throughout the completion of my thesis. I also want to thank my family and friends for always being there for me and supporting me throughout my academic journey.

Statutory declaration

I hereby declare that, this thesis entitled: **Classification of Turbofan Engine Degradation Using Machine Learning Methods on C-MAPSS Data** was written and completed by me. I also declare that all the sources and information used to complete the thesis are included in the list of references. I agree that the thesis could be made public in accordance with Article 47b of Act No. 111/1998 Coll., on Higher Education Institutions and on Amendments and Supplements to Some Other Acts (the Higher Education Act), and in accordance with the current Directive on publishing of the final theses. I am aware that my thesis is written in accordance to Act. 121/2000 Coll., on Copyright and therefore Mendel University in Brno has the right to conclude licence agreements on the utilization of the thesis as a school work in accordance with Article 60(1) of the Copyright Act. Before concluding a licence agreement on utilization of the work by another person, I will request a written statement from the university that the licence agreement is not in contradiction to legitimate interests of the university, and I will also pay a prospective fee to cover the cost incurred in creating the work to the full amount of such costs.

Abstract

This thesis presents the implementation of a machine learning classifier, specifically a Random Forest, which classifies turbofan engine health using the NASA C-MAPSS dataset. The study began with a review of existing predictive maintenance strategies and previous research on C-MAPSS data. This review revealed a distinct lack of discrete classification approaches for this specific problem. Therefore, this work focuses on shifting the traditional continuous prediction of Remaining Useful Life into a multi-class classification framework that better aligns with real-world maintenance decision-making.

During implementation, focus was on creating rational thresholds for engine health states, data preprocessing, and developing features using a sliding window, which allows feeding raw time-series data into a standard classification model.

As a result, a Random Forest classifier was built and demonstrates strong results in predictive performance. It successfully classifies minority classes while maintaining a low rate of false alarms.

Keywords

C-MAPSS, machine learning, time-series data, classification, Random Forest

Abstrakt

Tato práce představuje implementaci klasifikátoru strojového učení, konkrétně algoritmu Random Forest, pro klasifikaci zdravotního stavu dvouproudových motorů s využitím datové sady NASA C-MAPSS. Studie byla zahájena rešerší stávajících strategií prediktivní údržby a předchozích výzkumů aplikovaných na data C-MAPSS. Tato rešerše odhalila znatelný nedostatek přístupů využívajících diskrétní klasifikaci pro tento specifický problém. Z tohoto důvodu se tato práce zaměřuje na transformaci tradiční spojité predikce zbývající doby životnosti do rámce vícetřídní klasifikace, což lépe odpovídá reálnému rozhodování v oblasti údržby.

Během implementace byl kladen důraz na stanovení racionálních prahových hodnot pro jednotlivé stavy motoru, předzpracování dat a tvorbu příznaků s využitím metody plovoucího okna. Tento postup umožnil transformovat surová data z časových řad do formátu zpracovatelného standardními klasifikačními modely.

Výsledkem je sestavený klasifikátor Random Forest, který vykazuje vysokou prediktivní výkonnost. Úspěšně klasifikuje menšinové třídy při zachování nízké míry falešných poplachů.

Klíčová slova

C-MAPSS, strojové učení, data časových řad, klasifikace, Random Forest

Contents

Chapter 1

Introduction

During the preparation of this work, the Gemini 3 Flash [1] generative AI tool was utilized to assist with the formal formatting and structuring of bibliographic citations. To ensure accuracy and prevent model hallucinations, the full text of the cited articles was provided to the tool for processing.

1.1 Motivation

Modern turbofan engines are among the most complex and safety-critical assets in the engineering world. With the integration of advanced sensor networks, these engines have evolved into data-rich systems capable of transmitting their thermodynamic states in real-time.

Understanding this continuous sensor data is the core of Predictive Maintenance. In the aviation industry, switching from reactive or scheduled maintenance to a predictive approach provides huge benefits, such as preventing major engine failures, reducing unexpected repair costs, and making the process of ordering spare parts much more efficient.

Simulated datasets, such as NASA C-MAPSS [2], have generated significant academic interest in the research field. The motivation for this work stems from the conclusion that there is a lack of classification approaches for this specific dataset. The majority of research prioritizes regression problems for predicting the Remaining Useful Life (RUL) of turbofan engines. From an industrial point of view, the goal is often not to predict the residual life but to determine the need for a maintenance action at a given time window.

1.2 Objectives

The objective of this work is to develop a machine learning classification model that shifts an initial continuous regression problem to a multi-class classification problem. To achieve this, the following partial objectives are defined:

1. Develop discrete health state thresholds to categorize engine degradation into three actionable phases:
    - Safe: Normal operation range.
    - Warning: Degradation detected; maintenance planning required.
    - Critical: Immediate maintenance required

2. Data preprocessing and Feature Engineering
3. Classification Model implementation and evaluation

Chapter 2

Theoretical Background

2.1 Maintenance Approaches

Corrective Maintenance

Historically, the dominant strategy in industrial asset management was corrective maintenance, also known as reactive maintenance, where corrective action is taken only after the breakdown [3]. In the context of early industrialization, where machinery was mechanically simple, redundant, and separated from supply chains, this was a logical economic choice because the cost of maintaining a spare parts inventory and labor force was often lower than the implementation of complex monitoring regimes [3].

In the aviation sector, a purely corrective strategy is unacceptable for critical systems due to strict safety regulations. Applying it to a turbofan engine is impossible, as it allows catastrophic failure during a flight scenario that certification standards are designed to prevent entirely.

Preventive Maintenance

To reduce the risks associated with unexpected failures, Industry 2.0 adopted preventive maintenance, also known as planned maintenance, where the strategy is to schedule maintenance activities at predetermined intervals, regardless of the actual health status of the physical asset [3].

Preventive maintenance also introduces the waste of RUL. "RUL is the length of time a machine will operate before it requires repair or replacement" [4]. By estimating RUL, engineers can schedule maintenance, optimize operating efficiency, and avoid unplanned downtime [4]. Components are frequently replaced or fixed while they still carry a significant operational margin. This results in excessive consumption of backup parts and unnecessary labor costs.

Condition-Based Maintenance

The rise of the Internet of Things (IoT) and Supervisory Control and Data Acquisition (SCADA) [5] systems, which integrate interconnected devices, remote terminal units (RTUs), and communication networks to collect and process real-time data [5], has enabled the transition toward Condition-Based Maintenance (CBM) [6]. CBM utilizes continuous information regarding an equipment's condition to recommend timely and appropriate maintenance actions [6]. This proactive approach relies heavily on IoT systems to

monitor specific physical parameters in real-time, such as vibration frequencies, oil temperature, or pressure differentials. Because the primary objective of CBM is to prevent functional breakdowns and significant performance drops [6], maintenance is executed strictly when these monitored indicators show clear signs of decreasing performance or imminent failure.

Predictive Maintenance

Predictive Maintenance (PdM) provides a data-driven approach that utilizes IoT and machine learning. Unlike CBM, PdM is based on historical analysis of data and involves finding patterns to detect anomalies using machine learning methods. Machine learning algorithms are trained to recognize these patterns and predict failures based on historical data [7]. By combining operational data with real-time parameters monitored by aircraft sensors, such as air pressure, temperature, airspeed, and fuel flow, these models can successfully determine engine health, and appropriate maintenance can be scheduled at a suitable time [7].

Modern turbofan engines, such as Rolls-Royce turbofan jet engines, are equipped with a large number of IoT sensors that gather data used for maintenance predictions [8]. This approach is able to optimize the trade-off between safety and cost by enabling maintenance only when it is actually needed. Unlike preventive maintenance, which replaces components based on age or usage and often leads to unnecessary costs, PdM ensures maintenance is performed right on time [7]. This proactive strategy minimizes unexpected downtime, reduces the need for disruptive and costly emergency repairs, and enhances the overall safety and operational efficiency of the aircraft [7].

2.2 Gap Analysis

Most of the reviewed articles focused on regression problems where machine learning models predict RUL. A review of research on the NASA C-MAPSS dataset reveals that the majority of studies, for instance [9], [10], [11] - prioritize minimizing error metrics, such as Root Mean Square Error (RMSE), using complex Deep Learning models like LSTM, CNN, RNN. For practical deployment, this can lead to limitations such as a lack of interpretability, where those models are commonly referred to as black box models, offering little insight for maintenance [12]. If we examine real-world maintenance decisions, they are typically discrete (e.g., "Inspect," "Repair," or "Replace") rather than continuous. Providing an exact RUL estimate does not map directly to these binary or categorical operational decisions. A practical example of this discretization is the Automotive Oil Life Monitor (OLM) found in modern vehicle systems like Honda Maintenance Minder [13]. While the OLM algorithm continuously integrates engine data to calculate a remaining oil life percentage, the output to the driver is discrete: the system triggers specific maintenance codes, for example 'Code A1' for Oil Change + Tire Rotation, only when hard thresholds are reached, forcing a binary decision to service the vehicle [13].

Chapter 3

Health State Classification as Supervised learning problem

3.1 Supervised Learning

Supervised learning is a machine learning paradigm that focuses on learning an unknown target function. In this context, the target function f is unknown. However, the algorithm has a finite set of input-output pairs [14]. Using inductive reasoning[1], the model learns to infer the underlying mathematical rules that map inputs to their correct outputs.

Formally, the training dataset $\mathcal{D}$ consists of N training samples:

$$\mathcal{D} = \{(x_1, y_1), \ldots, (x_N, y_N)\} \tag{3.1}$$

where each $x_i \in \mathcal{X}$ represents an input or feature vector, and each $y_i \in \mathcal{Y}$ represents the corresponding target value. The task is to find a function $h_\theta(x)$, parameterized by a set of weights θ, that accurately maps the feature vector to the target value.

To learn the optimal mapping, we must define what it means for the model to be wrong. Most machine learning algorithms, such as neural networks or linear regression, utilize a Loss Function, as we denote $J(\theta)$. This function measures the error between the model's predictions and the actual ground truth values. The goal of the training process is to find the specific set of parameters θ^* that results in the lowest possible error throughout the data set.

$$\theta^* = \underset{\theta}{\operatorname{argmin}}\ J(\theta) \tag{3.2}$$

In regression problems, a common choice is the Mean Squared Error (MSE), which averages the squared differences between predictions and actual targets:

$$J(\theta) = \frac{1}{N}\sum_{i=1}^{N}(h_\theta(x_i) - y_i)^2 \tag{3.3}$$

Another approach in tree-based algorithms, such as the Decision Tree and Random Forest. These models rely on a splitting criterion, which evaluates how well the data is separated at each step of the learning process. For classification tasks, the goal is to

[1] Unlike deductive reasoning, which aims for certainty, inductive reasoning deals with probability, using specific examples to suggest a probable general conclusion

increase the homogeneity of the class labels in the resulting subsets. A commonly used measure is the Gini impurity, defined as

$$G = 1 - \sum_{k=1}^{K} p_k^2 \tag{3.4}$$

where p_k represents the proportion of samples belonging to class k. Lower values indicate that the data are more concentrated in a single class.

Another frequently used measure is entropy. Entropy is a measure of the uncertainty [15], in this case, in the class distribution:

$$H = -\sum_{k=1}^{K} p_k \log p_k \tag{3.5}$$

Again, lower values correspond to more homogeneous class assignments. For regression tasks, the objective is to reduce the spread of target values within subsets of the data. This is typically achieved by minimizing the Mean Squared Error (MSE), defined locally as:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \bar{y})^2 \tag{3.6}$$

where $\bar{y}$ denotes the average target value. These methods construct the model through a sequence of locally optimal decisions, each guided by a criterion that measures the quality of data separation

Optimization

Once the loss function is defined, the algorithm needs a way to efficiently search for the parameters θ that minimize it. For this task, there exists an iterative optimization algorithm called Gradient Descent. It works by calculating the gradient of the loss function. Gradient is a vector ∇J that gives the magnitude and direction of the steepest slope of the function [15]. To minimize the error, the algorithm updates the parameters in the opposite direction of the gradient:

$$\theta_{new} = \theta_{old} - \eta \cdot \nabla_\theta J(\theta) \tag{3.7}$$

Where, η is a learning rate or step size, which is a hyperparameter that controls the size of the steps the algorithm takes during optimization. If η is too small, the convergence will be very slow, and if η is too large, the algorithm may overshoot the minimum [15]. By repeating this update step iteratively, the model parameters eventually converge to a state where the loss is minimized.

Tree-based methods utilize a greedy algorithm, which at each step selects locally optimal decisions without considering their long-term global impact [16]. Once a decision is made, it is not adjusted later in the training process. This choice is made by evaluating all possible options and choosing the one that provides the greatest immediate improvement.

3.1.1 Generalization and Model Complexity

The ultimate goal of supervised learning is not only to perform well on training data but also on new, unseen data. This ability is known as generalization.

A central challenge in achieving good generalization is finding the right level of complexity of the model. If a model is too simple, it cannot capture the underlying patterns. If it is too complex, it captures the noise. This is often described using the concepts of Underfitting and Overfitting.

Figure 3.1: Different models based on level of generalization. Adopted from [17].

Underfitting

Underfitting occurs when the model is too simple to capture the underlying structure of the data. For example, trying to fit a linear curve to data that follows a quadratic curve. In the training process, it is characterized by high training error.

Overfitting

Overfitting occurs when the model is essentially "memorizing" the training data, including its random noise and outliers, rather than learning the general trend. The model creates an overly complex function that passes through nearly every data point.

The Bias-Variance Tradeoff

There is a natural tension between these two states. As we increase the complexity of a model, bias decreases, meaning it fits the training data better, but variance increases, which means it is less suited for unseen data. The objective is to find the good spot in the middle, a model that is complex enough to capture the true relationship, but simple enough to ignore the random noise.

3.1.2 C-MAPSS from NASA

Figure 3.2: Commercial turbofan engine. Adopted from [18].

Initially, the C-MAPSS package consists of four datasets: FD001, FD002, FD003, FD004. Each dataset represents a simulated, in MATLAB [19] and SIMULINK [20] environment, sensor measurements of a standard commercial turbofan engine [2], which is a complex gas turbine system designed to generate thrust by accelerating air [18]. Dataset summaries are presented in Tables 3.1 - 3.4, which were taken from the README file in the C-MAPSS package, but with an additional number of samples (rows).

Let us describe what is meant by operational setting and fault mode from a health state classification perspective. An operational setting in the dataset means specific flight and environmental conditions under which the engine operates [2]. From our perspective, the challenge is that the model must learn to separate expected sensor variations caused by changing flight conditions from abnormal sensor variations representative of an actual engine health problem. A fault mode in the dataset is an unspecified issue, which represents the specific type of physical degradation with an increasingly worsening effect [2]. For instance, FD003 consists of 21 sensor measurements, summarized in Table 3.5. They describe the physical state of the engine: physical meanings, units, and expected degradation trends. It is important to note that the table, which was adopted from [21], does not mention the 3 operational settings in the table.

Our choice of the FD003 dataset from the C-MAPSS package was made for these reasons: this specific subset offers a balanced challenge for models, unlike datasets with a single fault mode, it introduces complexity by simulating degradation in two distinct components simultaneously, however, it has a single operating setting, whereas multiple settings could significantly increase complexity. So we chose FD003 because of its balanced complexity.

Table 3.1: FD001

Parameter	Value
Operating Conditions	1
Fault Modes	1
Training Samples	20,631
Training Units	100
Test Units	100

Table 3.2: FD002

Parameter	Value
Operating Conditions	6
Fault Modes	1
Training Samples	53,579
Training Units	260
Test Units	259

Table 3.3: FD003

Parameter	Value
Operating Conditions	1
Fault Modes	2
Training Samples	24,270
Training Units	100
Test Units	100

Table 3.4: FD004

Parameter	Value
Operating Conditions	6
Fault Modes	2
Training Samples	61,249
Training Units	248
Test Units	249

Table 3.5: Physical meanings of the sensors measurements. Adopted from [21].

Sensor	Symbol	Description	Units	Trend
1	T2	Total temperature at fan inlet	°R	$\sim$
2	T24	Total temperature at LPC outlet	°R	$\uparrow$
3	T30	Total temperature at HPC outlet	°R	$\uparrow$
4	T50	Total temperature at LPT outlet	°R	$\uparrow$
5	P2	Pressure at fan inlet	psia	$\sim$
6	P15	Total pressure in bypass duct	psia	$\sim$
7	P30	Total pressure at HPC outlet	psia	$\downarrow$
8	Nf	Physical fan speed	rpm	$\uparrow$
9	Nc	Physical core speed	rpm	$\uparrow$
10	epr	Engine pressure ratio	–	$\sim$
11	Ps30	Static pressure at HPC outlet	psia	$\uparrow$
12	Phi	Ratio of fuel flow to Ps30	pps/psi	$\downarrow$
13	NRf	Corrected fan speed	rpm	$\uparrow$
14	NRc	Corrected core speed	rpm	$\downarrow$
15	BPR	Bypass ratio	–	$\uparrow$
16	farB	Burner fuel–air ratio	–	$\sim$
17	htBleed	Bleed enthalpy	–	$\uparrow$
18	Nf_dmd	Demanded fan speed	rpm	$\sim$
19	PCNfR_dmd	Demanded corrected fan speed	rpm	$\sim$
20	W31	HPT coolant bleed	lbm/s	$\downarrow$
21	W32	LPT coolant bleed	lbm/s	$\downarrow$

3.1.3 Classification Task

Classification is defined as a predictive modeling technique used to predict group membership for specific data instances [22]. In the context of supervised learning, the objective is to optimize a function or model that maps input features to class labels. There are basically two types of classification tasks: binary classification, where we assign instances to one of two possible classes, and multiclass classification, where we assign instances to one of more than two classes. Therefore, health state classification is a multiclass classification.

Time-Series Classification

Time-Series Classification (TSC) is a special classification task in machine learning, where the data is a time-ordered sequence of values, where the relative position of each data point carries critical information that is often more significant than the individual values themselves [23]. The challenge in fact is that standard classification algorithms, which are designed for structured data or assume the independence between each feature, such as the Naive Bayes algorithm, are not well-suited for time-series data [23]. We cannot feed such data into these models. Therefore, we must find a method to process the data in a way that makes it suitable to feed into the model. There are various techniques for dealing with time-series data. Irani et al. [24] discuss different approaches for solving the TSC task.

Irani et al. [24] made a review on so-called time-series embeddings, which is a technique for representing time-series data in the form of feature vectors (vector embeddings). Applying embedding methods can be very useful in the context of time-series data, because such data is often high-dimensional, and embedding can reduce this dimensionality while preserving essential information from the data [24]. Another useful approach involves Feature extraction, which is part of Feature Engineering, in which we transform features or create new features to enhance the performance of the algorithm, and this can be done automatically or manually by some of the embedding methods [24]. Basically, we can separate two groups of methods: first , those that extract features from time-series that can then be fed into the standard classifier, second, those that work on raw time-series [23]. Since our objective is to train a standard classifier, we describe the first group. Irani et al. [24] call the first group Feature-Based, and also categorize into two groups:

- Hand-Crafted - manually extract statistical measures [24]. We manually define what features we want to extract, for instance: STD, mean, slope, min-max values in a fixed-size window, where the window is a segment of data points extracted from the total sequence. From the C-MAPSS perspective, it would be an engine state over the last N cycles.

- Automated - using specific libraries like catch22 [25] or TSFRESH [26]. Irani et al. [24] suggest that they are useful in the case of limited domain knowledge. Based on research of those libraries, catch22 fits the dataset for those reasons: TSFRESH extracts over 700 features per time-series by default, and applying this across a sliding window expands the feature space exponentially, risking severe overfitting on the dataset. In contrast, catch22 extracts exactly 22 features, which is not so overwhelming a model. Also, implementing a sliding window approach across thousands of trajectories has a computational cost. Catch22 relies on a highly optimized C-

backend, executing significantly faster and with a much smaller memory footprint compared to the extraction of TSFRESH.

3.2 Model Evaluation Metrics

In classification tasks, a very important part is choosing the right evaluation metric, especially in cases where the dataset might be imbalanced.

Confusion Matrix

A confusion matrix is a matrix that is used as a tool for quantifying the performance of a classification algorithm [27]. By plotting the confusion matrix after training, we can easily see the number of predicted True Positive (TP), True Negative (TN), False Negative (FN), False Positive (FP). In multi-class classification, we count TP, TN, FN and FP for each class separately.

Table 3.6: Confusion Matrix

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

Metrics

Accuracy is the first widely used, simplest metric. The ratio of accurately classified data items to the total number of predictions. The accuracy, in the case of an imbalanced data, is not a good choice, because the model can show very high accuracy and still miss the minor class [28, 29].

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.8}$$

Precision measures the accuracy of positive predictions among all positive predictions, no matter if it is a true positive or a false positive [29]. Simply means if the model predicts that an engine health state is in the Warning class, how confident can we be that this is true.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.9}$$

Recall measures the accuracy of positive predictions among all true instances of positive [29]. This simply means that if an engine is actually in the Warning class, what is the probability that the model will successfully detect it.

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{3.10}$$

Let us project these interpretations to the problem of engine health classification. For example, imagine the Warning class has high Precision and low Recall. In this case, most of these Warning predictions will be correct, if the classifier flags an engine as Warning, the probability that it is actually entering the degradation phase is very high.

The problem is that there are many other actual Warning engines that the algorithm missed and incorrectly assigned. This incorrect assignment can be clearly visualized in the confusion matrix.

On the other hand, low Precision and high Recall for the Warning class means the model captures almost all actual Warning engines, but due to low Precision there will be many healthy "Safe" engines misclassified.

Precision and Recall are more suitable metrics for our classification problem because we have a highly imbalanced dataset. The metric F-score takes both into consideration by calculating the harmonic mean of these values [29].

$$F - score = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.11}$$

3.3 Machine Learning Algorithms

Random Forest

Random Forest is an ensemble machine learning algorithm. Ensemble means that an algorithm inside utilizes multiple models. In the case of Random Forest, it is a Decision Tree. Decision Tree is a hierarchical predictive model that maps data observations through a series of branches and nodes to reach a specific terminal result. That mirrors human logical deduction, so this technique is considered transparent and interpretable for classifying complex datasets [30].

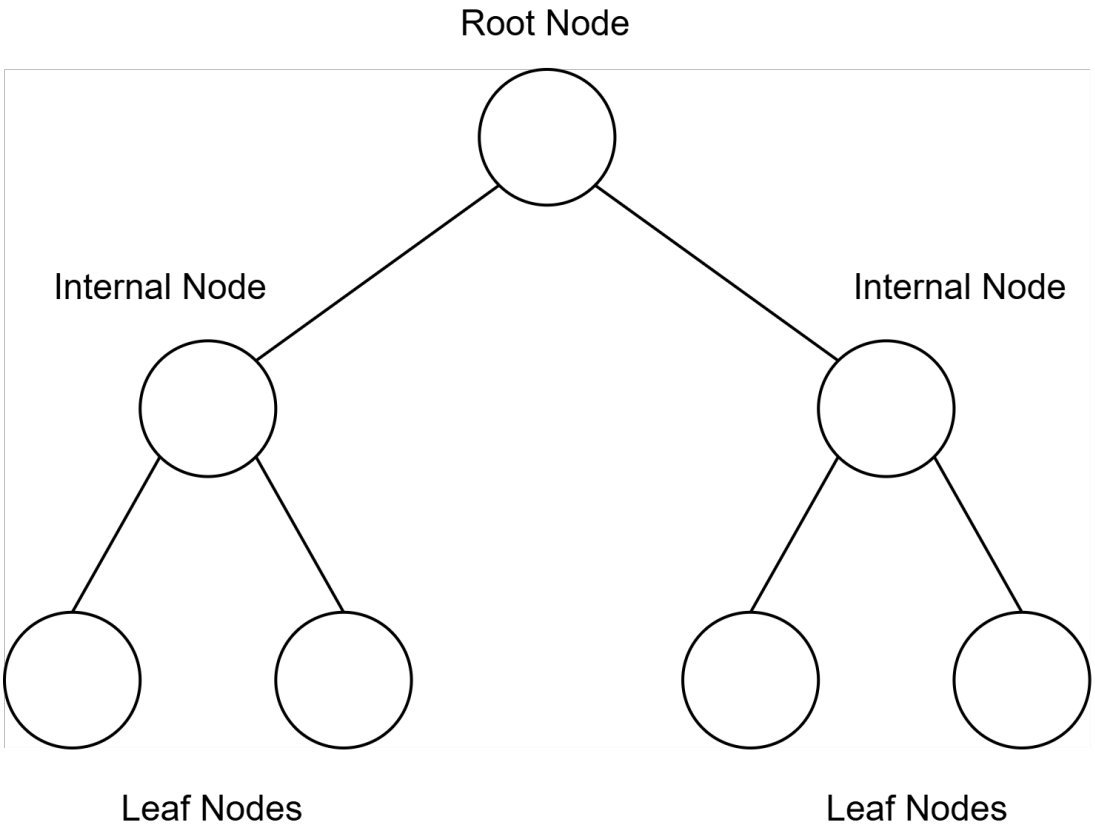


Figure 3.3: Decision Tree

Random Forest works by aggregating an ensemble of decision trees to minimize data correlation and improve predictive accuracy. The algorithm achieves this through two layers of randomization. The first is Sample Randomization, which selects random subsets of data from the training set. The second is Feature Randomization, which chooses a random subset of features for each tree [30].

By implementing these layers of randomization, Random Forest offers several advantages [30]:

- Unlike individual trees that may become too complex, the ensemble average effectively cancels out noise, which prevents overfitting.
- With a linear time complexity the algorithm is well suited for large-scale datasets.
- The model provides variable importance (feature importance), which can serve as a means of interpretation and validation.

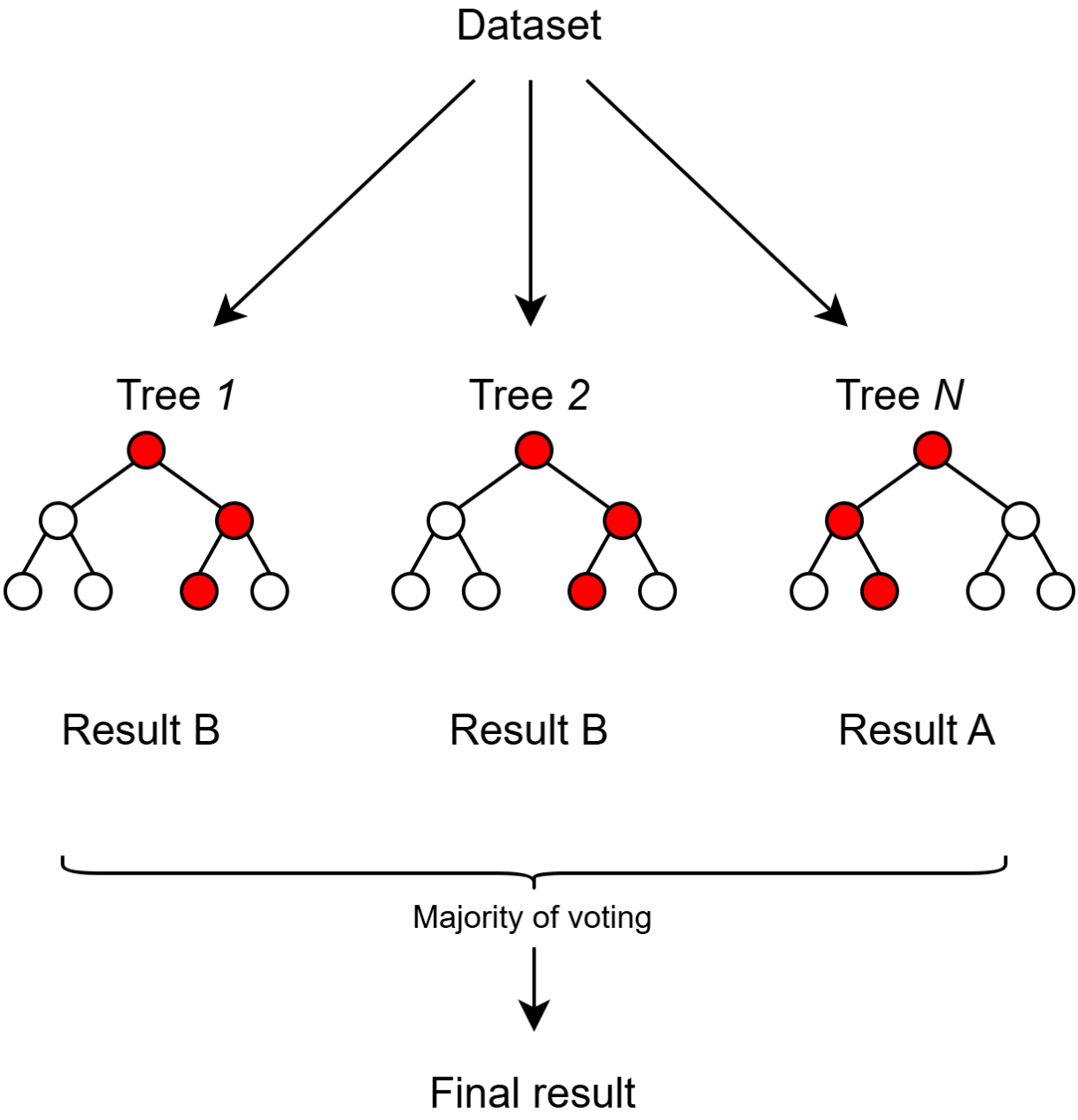


Figure 3.4: Random Forest workflow

Chapter 4

Methodology

4.1 Existing Approaches

In this chapter, we propose a method for solving this classification problem. Currently, very few researchers have engaged with the gap discussed in Section 2.2 by proposing discrete health-state architectures.

For instance, Chaibi et al. [31] redefined the problem of predicting RUL as a multi-class health classification problem with three classes. These classes are generated based on a Life Ratio (LR), which is the ratio between the current operating cycle and the total lifetime in cycles [31]. Specifically, health states (classes) are defined into a Healthy State ($LR \leq 0.6$), a Moderate State ($0.6 \leq LR \leq 0.8$), and an Alert state ($LR \geq 0.8$) [31]. Utilizing gradient boosting algorithms such as CatBoost [32], XGBoost [33], and Random Forest, their conclusions suggest that CatBoost provides a more balanced performance across degradation states compared to XGBoost and Random Forest, achieving an overall accuracy of 70%. While CatBoost demonstrated a high Precision of 0.89 and Recall of 0.79 for the healthy state (Class 0), for the moderate state (Class 1), it is 0.39 for and 0.72 for the alert state (Class 2). Recall for these critical transitions was at 0.63 for (Class 1) and 0.52 for (Class 2).

Soni et al. [34] evaluated various models but chose Random Forest as the best performing model. In this case, labels are defined through trial and error [34]. This approach achieved a test accuracy of 81%. The model shows effectiveness in identifying healthy engines, achieving a Recall of 0.90 for the good condition and a precision of 0.79. However, these works show a significant loss of Recall when differentiating between moderate degradation and failure, in our case, it would be Warning and Critical, often resulting in misclassification between neighboring classes. In the work [31], 1,040 instances of the alert state (Class 2) were misclassified as moderate degradation (Class 1), while 505 instances of the moderate state were confused with the healthy state.

Class naming

To maintain consistency throughout this work, we map the health-state terminology used in related studies to our proposed class naming convention. In our work, the classes are defined as Safe (Class 0), Warning (Class 1), and Critical (Class 2). These correspond conceptually to the Healthy, Moderate, and Alert states defined by Chaibi et al. [31], as well as the good condition, intermediate degradation, and failure-related states discussed by Soni et al. [34]. Although the naming differs between studies, the underlying objective

remains the same: distinguishing between normal operation, early degradation requiring maintenance, and severe degradation requiring immediate maintenance action.

Our approach

To overcome these limitations in existing studies, our methodology introduces cross-validation, feature selection, and engineering, an approach to handling class imbalances and specifically maximizing the Precision of classifying Warning class, which means minimizing false alarms.

4.2 Use Case

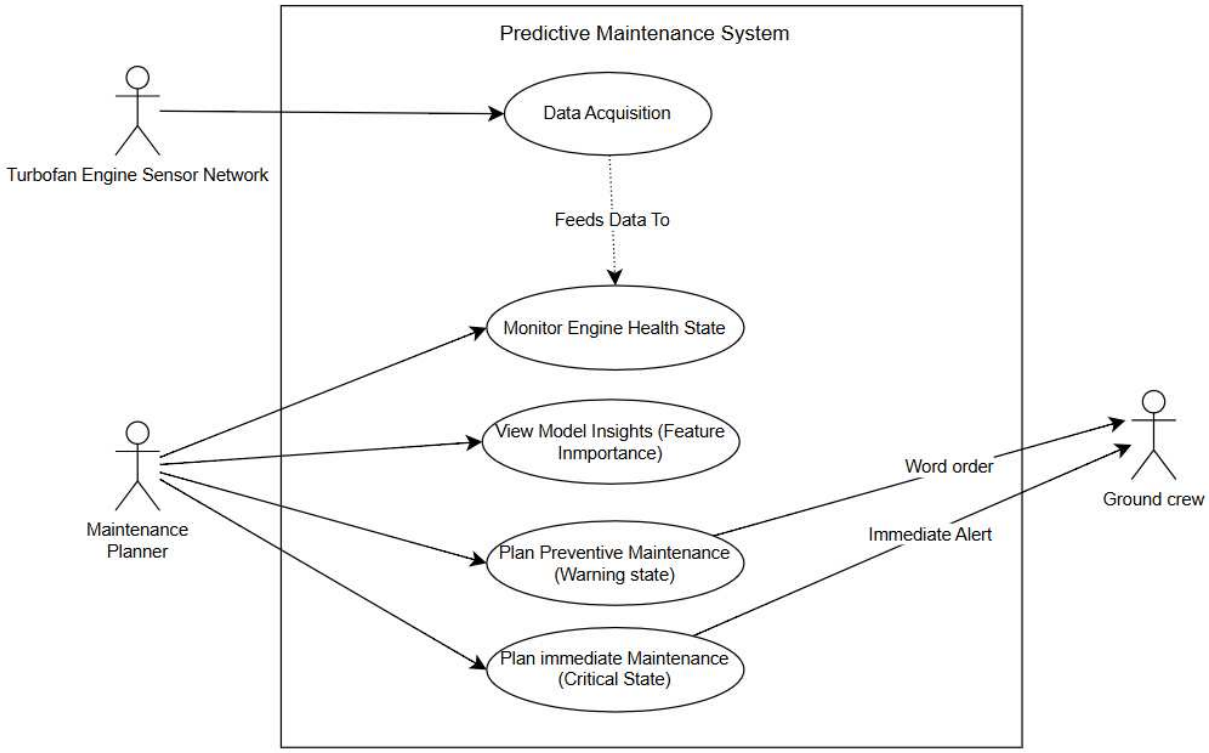


Figure 4.1: Use case diagram of predictive system

The Unified Modeling Language (UML) Use Case diagram, presented in Figure 4.1, illustrates the proposed application of the model that will result from this work. The diagram defines the boundaries of the Predictive Maintenance System and illustrates the interactions between external actors and core internal processes. The diagram demonstrates how sensor data is transformed into maintenance actions.

The architecture defines three distinct actors within the operational environment:

- Turbofan Engine Sensor Network: This entity represents the sensor network on the aircraft. Its primary function is data acquisition. As shown by the dependency arrow, the data collected by this network directly feeds the central Monitor Engine Health State module.

- Maintenance Planner: The planner utilizes the system interface to track engine degradation and can see model interpretability through Feature Importance analysis. Based on the discrete health classifications generated by the model, the planner can execute Plan Preventive Maintenance for engines entering the Warning state, or trigger Plan immediate Maintenance for engines identified in the Critical class.

- Ground Crew: This actor represents the physical technicians. They act as the receivers of the system's outputs. Depending on the planner's actions, the ground crew receives either standard Work orders for scheduled preventive tasks or Immediate Alerts necessitating emergency inspections.

4.3 Proposed Classification Models

Random Forest

Reasons for choosing Random Forest:

First, the algorithm handles non-linear relationships and complex data interactions [30]. Because engine degradation is rarely purely linear, the underlying decision trees can naturally learn specific operating thresholds while remaining highly resilient to noisy data and outliers [30]. Second, the algorithm performs feature selection during the training process [30]. By calculating the Mean Decrease Impurity (MDI), which measures the overall reduction in node impurity when splitting on a specific variable, the model can automatically evaluate variable importance [30]. This allows the algorithm to naturally ignore irrelevant sensors or noise that may have survived the preprocessing phase. This feature importance metric also serves as an interpretable tool for maintenance planners. By revealing exactly which sensor readings are driving the degradation predictions, it allows engineers to trace the model's mathematical logic back to specific physical components, enabling transparent maintenance actions. Third, it can handle large, high-dimensional datasets without slowing down or losing accuracy [30]. Finally, Soni et al. [34] showed Random Forest as the best performing algorithm for this problem, compared to other existing approaches, such as [31].

Random Forest Optimization Strategy

Random Forest performance can be significantly enhanced through hyperparameter optimization. Achieving the best comparative results requires a combination of hyperparameter tuning and effective cross-validation techniques [30]. So, to optimize the Random Forest model for our specific dataset, we should define a tuning strategy. This involves selecting the most influential parameters, understanding their direct impact on the training process, and adjusting them through an automated, in the case of cross-validation, tuning method.

We suggest testing the following parameters [30]:

- Number of Trees: The total number of decision trees utilized within the forest ensemble.

- Stopping Criteria: The minimum number of samples required to reach a node before the algorithm stops further splitting

- Tree height: The maximum depth of each tree.

We can split parameters that we decide to tune into two categories, based on their impact on the model [30]:

- Category-A Parameters: Increasing the values of these parameters creates a more powerful model but increasing them too far can overfit (Number of Trees, Tree height).
- Category-B Parameters: Increasing the values of these parameters results in a less powerful model and the model can underfit (Stopping Criteria).

4.4 RUL Calculation and Label Generation

Because we are dealing with supervised learning, we should define a method of creating target values, which raw data does not contain, we know only how many cycles each engine lived. First, the RUL value will be calculated for every engine as:

$$RUL = \text{Total Life of Engine} - \text{Current Cycle}$$

After this, we can define class boundaries. For instance, Soni et al. [34] did not explicitly define class boundaries and instead used a trial-and-error approach to maximize model accuracy. Yildirim et al. [35] define the class boundaries based on typical industrial practice, where the last 10 cycles represent failure requiring immediate action, cycles 10–20 indicate degradation that warrants increased monitoring, and cycles 20 - 30 represent normal operation. Chaibi et al. [31] define the class boundaries based on a Life Ratio, discussed in Section 4.1.

We suggest boundaries based on typical industrial practice defined by Yildirim et al. [35], but with an expanded zone for Warning and Safe classes: Class 0 (Safe): $RUL \geq 30$ - normal operating condition. Class 1 (Warning): $10 \leq RUL < 30$ - the primary target, representing the critical window for maintenance. Class 2 (Critical): $RUL < 10$ - imminent failure requiring immediate action.

4.5 Preprocessing

Data preprocessing is an important phase of the machine learning pipeline as it can significantly optimize the training process and ensure the model is able to extract meaningful patterns from the raw time-series data. Raw sensor data are rarely suitable for direct input into machine learning algorithms. The dataset contains noise and redundant information that can severely diminish model performance. Also, a very common step in the preprocessing pipeline of sensor data, where features represent different physical measurements, is normalization. We suggest skipping the normalization phase, as Random Forest is generally not as sensitive compared to a distance-based algorithms [30].

Data Cleaning

In the dataset, not all of the attributes carry useful information regarding engine health. Some of them can be constant or near constant. We can identify this using the Standard deviation (STD). STD tells us how much the sensor readings change over time and is useful in this case.

Formally, the STD defined as:

$$STD = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2} \tag{4.1}$$

Some sensors in the dataset are known to experience STD of zero (constant) or near-zero (noise).

Feature Selection

Random Forest has a built-in feature selection mechanism, but to eliminate most irrelevant features, we will perform a correlation analysis between the remaining sensors and the calculated RUL. For the analysis of linear relationships, we can use the Pearson correlation coefficient [36], which will measure the strength of the linear relationship between each sensor signal and RUL.

The coefficient r is defined as:

$$r = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_i (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_i (X_i - \bar{X})^2}\sqrt{\sum_i (Y_i - \bar{Y})^2}} \tag{4.2}$$

The Pearson correlation coefficient ranges from -1 to 1, with 0 indicating no linear correlation [36]. Since both strong positive and negative correlations can indicate predictive power, we also consider the absolute correlation, defined as:

$$|r| = |\text{Correlation with RUL}| \tag{4.3}$$

The choice of the correlation threshold depends on how many features we want to keep. Lower thresholds keep more features, including weakly related ones, while higher thresholds remove more features, but may also remove useful degradation information.

We suggest using the threshold of $|r| \geq 0.3$, because values around 0.3 are commonly considered to represent at least a moderate linear relationship between variables [37]. This helps remove irrelevant features while still keeping potentially useful features for the model.

4.6 Feature Engineering

In Section 3.1.3 we discuss the approach for feeding time-series data to the standard model and make a point on two methods: Hand Crafted and using a specific library (catch22). Due to the benefits of the automated method using catch22, discussed in Section 3.1.3, we suggest utilizing this method.

Next, we should define the window size. This is an important part, as the chosen window size can have a significant impact on the performance of the model [38]. Notably, Wahid et al. [38] suggest the window size of $w = 30$ outperforms other configurations, so we choose that window size.

The process of extracting the following: for each window sequence, the algorithm extracts 22 features. These features represent diverse mathematical properties, for instance, linear autocorrelation, distribution shape, and incremental differences [25].

4.7 Training and Validation Strategy

In this phase, the final objective is to build a Random Forest Classifier that prioritizes Precision in classifying states of engine health. By this, we minimize false alarms or false positives.

Cross-Validation

Cross-validation is a fundamental technique primarily used to assess how the model generalizes to an independent data set [39]. By partitioning the data into training and testing sets, cross-validation helps in achieving a bias-variance tradeoff, discussed in Section 3.1.1. Standard cross-validation methods include K-folds cross-validation, which divides the data set into into equal-sized groups, and Repeated K-folds, which averages multiple runs to reduce variance [39]. Applying these traditional techniques to time-series data presents challenges, because standard methods assume that each sequence in the dataset is independent and identically distributed [40]. Applying common cross-validation methods to sequential data can lead to overly optimistic estimates of the model's performance.

Cross-Validation and Out-of-Bag Estimation

A specific property of the Random Forest is its internal validation mechanism, which can theoretically render traditional cross-validation redundant for basic error estimation. During the training phase, the Random Forest employs an ensemble technique known as Bootstrap Aggregating, or Bagging [30]. The process of selecting a training data set involves randomly choosing a subset of data points, with the potential of picking the same data point multiple times (sampling with replacement), from the initial set of data [30]. Every tree in the Random Forests algorithm is constructed using this randomly chosen portion of the data, which is typically a bootstrap sample that is 0.632 times the size of the original sample [30]. The observations that are not utilized in the construction of a specific tree are called out-of-bag (OOB) observations. Because the forest is constructed using training data, each tree is evaluated using the remaining 36.8% of the samples that were not used to create that specific tree [30]. This out-of-bag error estimate serves as a measure of the error in the random forest model while it is being built. Therefore, Out-of-Bag can serve as validation or test data, meaning that the model does not strictly require a testing dataset for result validation. This provides the algorithm with a built-in estimate of the generalization error.

Validation Strategy

Despite the computational efficiency of the built-in OOB estimation, relying solely on this metric presents limitations when optimizing the model. Hyper-parameter tuning is common method to make results better. To eliminate the data leakage, we should apply method that consider split by group, in our case group is one engine trajectory, using the unit number (engine ID) as the grouping key.

Optimal Data split

Data split ratio is defined by the amount of data from initial dataset that is separated for training and test set, exclusively for testing and evaluation of the model performance. The most common splits are 70/30 or 80/20. The simple intuition: if we leave more

data for training, the model generalizes better, but we get less data for testing, meaning our evaluation metrics are less reliable. If we leave less data for training, the model can overfit, but the testing evaluation will be highly reliable.

In case of applying split using engine id, we assume N engines for training and N for testing. We suggest a split 70/30 because it ensures more reliability of the model, where we validate and test on 30 engines, not only on 20, in the case of 80/20. The final train test valid split:

- Training Set: 70% utilized for model training and oversampling.
- Validation Set: 15% for tuning.
- Test Set: 15% for the final performance evaluation.

4.8 Addressing Class Imbalance

In supervised learning, class imbalance is a situation where the number of data instances in one class significantly outnumbers the instances in other classes. In the dataset, turbofan engines are simulated from a completely healthy state all the way down to system failure. Therefore, the engine operates normally for the vast majority of its life, and the rapid degradation or critical phase only happens at the very end. So we have a highly imbalanced dataset, where the Safe class is much larger than the Warning or Critical classes.

Even if we utilize the right metrics, discussed in Section 3.2, for classification evaluation, the model still needs enough data for training, to be able to learn underlying patterns well and avoid underfitting. Various techniques exist for dealing with imbalanced data.

Oversampling

Oversampling is a technique where we increase the number of instances in the minority classes. We can categorize two groups of oversampling methods:

- Random Oversampling: duplicating existing instances from the minority class.
- Synthetic Oversampling: in this case, the class imbalance is addressed by synthesizing new instances of the minority class. This is achieved by selecting a minority data point, identifying its k nearest neighbors within the same class, and generating artificial data points along the vectors connecting them [41].

The synthetic approach better fits to the dataset for those reasons: it balances the classes so the classifier is forced to pay equal attention to the Critical class, and it introduces slight variations into the data. From a C-MAPSS perspective, this means the model learns a larger representation of what engine failure looks like across different trajectories, improving its generalization.

Weighted Random Forest

Another approach is to adjust the class weights within the model. Class weight adjustment or cost-sensitive learning [42] works by modification of the algorithm calculations to prioritize specific classes. In cost-sensitive learning, it is done by assigning a higher weight to a minority class, which increases the penalty for misclassifying it. This forces

the model to become more aggressive in detecting that class, which generally increases its Recall. However, this standard "balanced" approach often comes at the cost of a severe drop in Precision.

Precision trade off

To specifically optimize Precision for the Warning class, custom class weights can be applied instead of standard balanced weighting. By assigning the Warning class a relatively lower weight than competing classes, the model becomes more conservative when predicting this state. During tree construction, splits that incorrectly classify Safe samples as Warning become more costly, effectively requiring stronger evidence before assigning a sample to the Warning class. By implementing this strategy, we expect to reduce false positives (false alarms) Warning predictions, and improve Precision.

Chapter 5

Implementation

This chapter details the implementation of the Random Forest classification model proposed in the methodology. It covers the software environment, data processing techniques, training, and evaluation. Important to note that the sections go in the same sequence as cells in a Python notebook, and considering the sequence without data leakage. A high-level overview of the proposed pipeline is illustrated in Figure 5.1

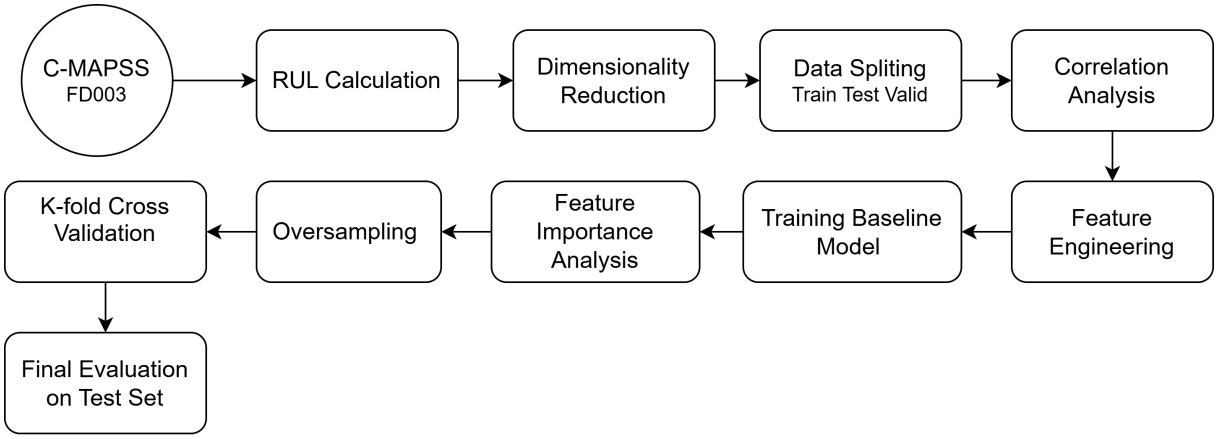


Figure 5.1: Machine learning pipeline

5.1 Development Environment and Libraries

We will utilize Python programming language, selected for its extensive ecosystem of data science libraries and robust support for matrix operations. The implementation relies on the following core libraries:

- Pandas: Utilized for performance manipulation with tabular data. Also provides the DataFrame structure necessary for handling time-series data [43].
- NumPy: For numerical computations and array operations. Very efficient in the case of processing C-MAPSS because the library offers a vectorized approach - utilization of C-based looping methods, instead of traditional for loops. [44].
- Scikit-Learn: for data preprocessing, dataset splitting, model training, and performance metric calculation [45].

Loading Strategy

It is critical to note that we utilize the train file in FD003 as the main dataset. Unlike the provided test file, which contains censored data, initially it was for a regression problem, the training file provides complete paths from a healthy state to system failure for every unit, so utilizing this dataset, we can calculate our own RUL for each unit and then define labels.

Global Configuration Parameters

Before executing the data preprocessing and training pipelines, we define a set of global constants to configure the behavior of the entire implementation. This ensures that all subsequent operations remain consistent and easy for debugging.

First, we define the critical configuration `SEED = 42`. Machine learning algorithms rely heavily on pseudo-random numbers, and as they are pseudo-random but not truly random, they need some starting point for calculation. The specific integer 42 chosen for the random seed is arbitrary, and basically any integer value can be used. By feeding the random seed to the core of NumPy, we guarantee reproducibility, ensuring that every compilation of the code produces the exact same algorithmic behavior.

Also, we define the data partitioning through the `TRAIN_SIZE = 0.70` constant. Next, we define the `WINDOW_SIZE = 30` parameter, which we will utilize in time-series feature engineering. Finally, we define `SELECTION_THRESHOLD = 0.30` for Pearson correlation feature selection.

5.2 RUL Calculation and Target Label Implementation

Remaining Useful Life Computation

Since the raw dataset does not explicitly provide RUL labels, they must be derived from the available history of engine life. For every engine unit in the training set, the maximum observed number of operating cycles represents its total lifetime (failure point).

Let $C_{\max}^{(i)}$ denote the total lifespan (maximum cycles) of engine i, and let $C_t^{(i)}$ denote the current operational cycle of that engine at a specific time step t. The RUL is formally defined as:

$$\mathrm{RUL}t^{(i)} = C\max^{(i)} - C_t^{(i)} \tag{5.1}$$

To implement this logic programmatically across the dataset, we utilize the `pandas` library. Rather than calculating this iteratively, which is computationally expensive, we execute a vectorized mapping:

```
max_cycles_df = df.groupby('unit_number')['time_in_cycles'].
    max().reset_index()
max_cycles_df.columns = ['unit_number', 'max_cycle']
df = df.merge(max_cycles_df, on='unit_number', how='left')
df['RUL'] = df['max_cycle'] - df['time_in_cycles']
```

RUL calculation

Labels generation

Class boundaries (labels), as discussed in 4.4, are defined in those ranges:

$$\text{Label} = \begin{cases} 0 & \text{(Safe)}, \quad \text{RUL} \geq 30 \\ 1 & \text{(Warning)}, \quad 10 \leq \text{RUL} < 30 \\ 2 & \text{(Critical)}, \quad \text{RUL} < 10 \end{cases} \tag{5.2}$$

We utilize the `numpy` and `pandas` libraries and apply a vectorized approach:

```
conditions = [
    df["RUL"] >= 30,
    (df["RUL"] >= 10) & (df["RUL"] < 30),
    df["RUL"] < 10,
]
df["label"] = np.select(conditions, [0, 1, 2])
counts = df["label"].value_counts().sort_index()
```

Creating label column based on conditions that include our class boundaries

After defining the label feature in the dataset, we can visualize the presence of class imbalance, discussed in Section 4.8, on the plot:

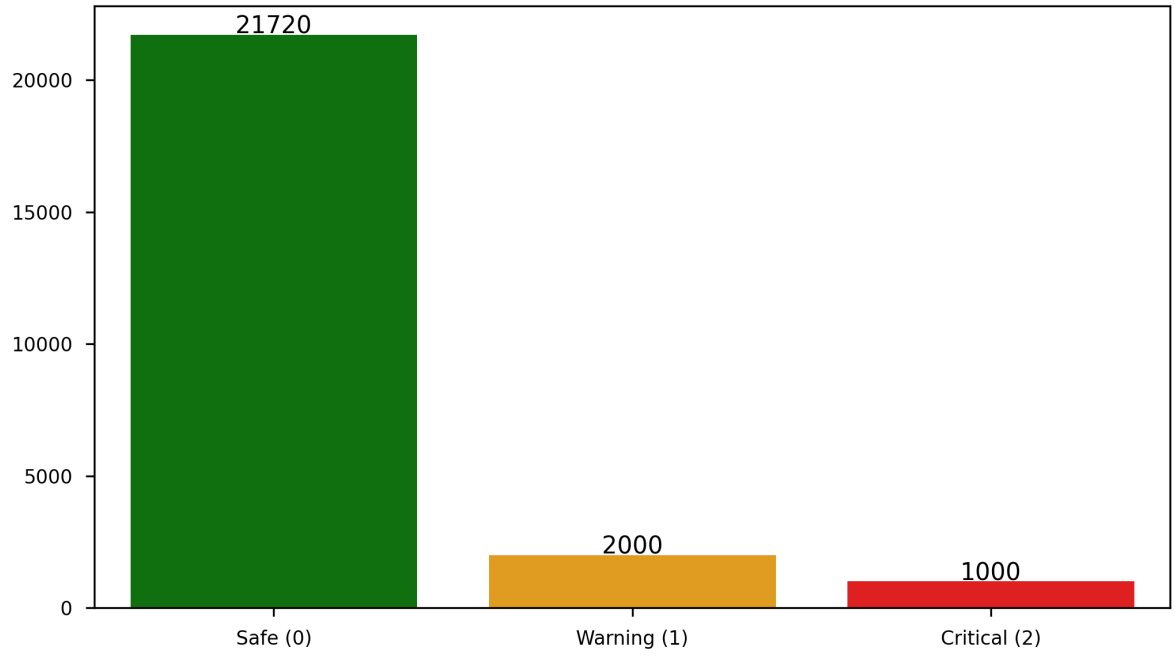


Figure 5.2: Class distribution

5.3 Preprocessing

In this section, we programmatically execute the data preprocessing methods theoretically outlined in Section 4.5.

Dimensionality Reduction

The first step is identifying and eliminating non-informative features. Several sensors and operational settings record constant values throughout the entire simulated lifespan of the engines. Because these signals exhibit STD of zero or near zero, they have no discriminative power for health-state classification and merely artificially inflate the dimensionality of the feature space.

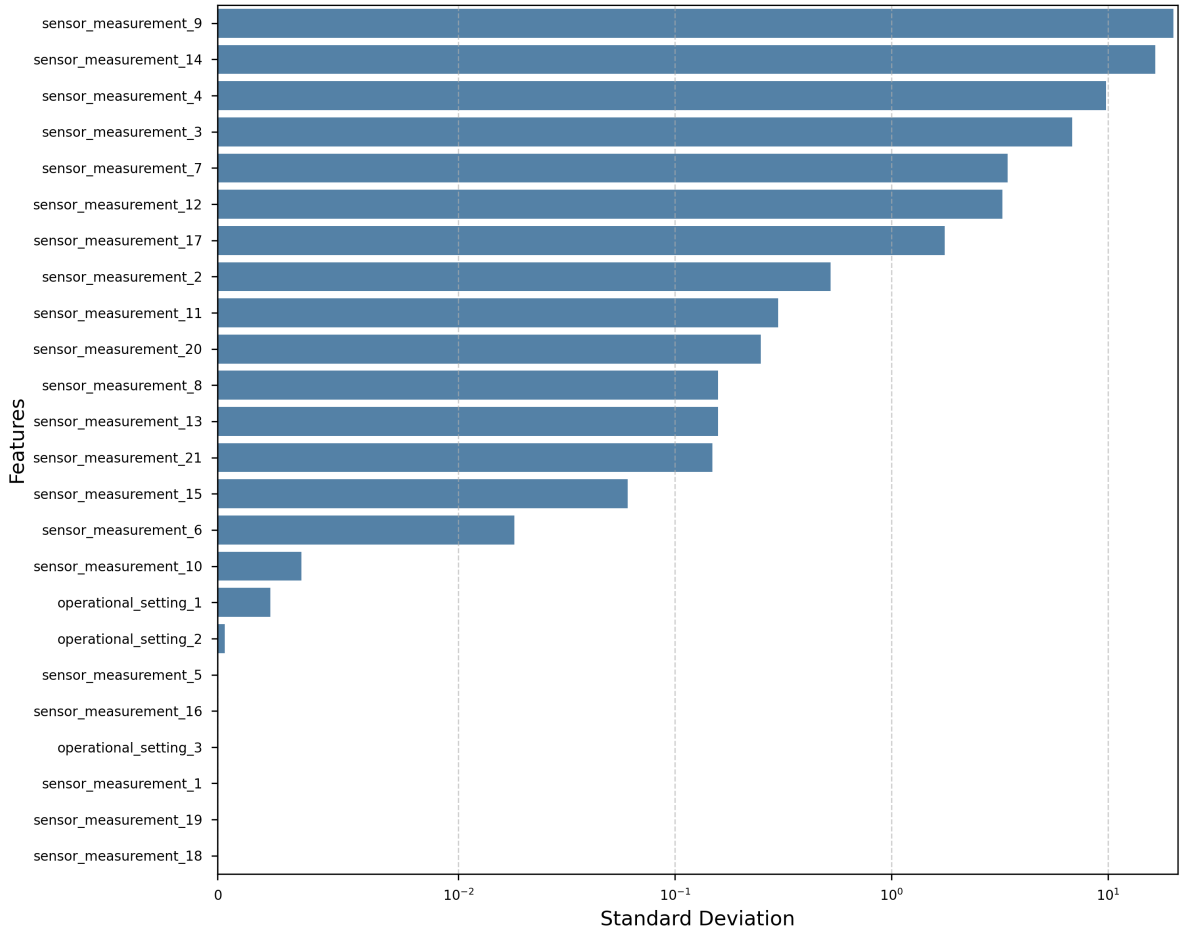


Figure 5.3: Sensor variability analysis.

We compute the STD across all columns in the dataset and if column demonstrating zero or near zero variance, we subsequently drop them from the dataframe:

```
sensor_setting_columns = [c for c in df.columns if "sensor" in
    c or "setting" in c]
std_values = df[sensor_setting_columns].std().sort_values(
   ascending=False)

constant_columns = std_values[std_values < 0.001].index.tolist
   ()
df.drop(columns=constant_columns, inplace=True)
```

Dropping zero or near zero variance columns from dataset

As a result of this operation, multiple columns are removed from the dataset. This step cleans the data, so we can efficiently utilize the feature extraction algorithm catch22.

Splitting Dataset

The dataset is divided based on the unit number, ensuring that all chronological cycles belonging to a specific engine remain entirely within a single partition.

```
groups = df['unit_number']
```

Defining group by unit number of engine

Split into 70% Train.

```
splitter_train = GroupShuffleSplit(n_splits=1, train_size=
    TRAIN_SIZE, random_state=SEED)
train_idx, temp_idx = next(splitter_train.split(X, y, groups))
X_train, y_train = X.iloc[train_idx], y.iloc[train_idx]
X_temp, y_temp = X.iloc[temp_idx], y.iloc[temp_idx]
groups_temp = groups.iloc[temp_idx]
```

Implementation of splitting dataset into train using

Split into 15% Val and 15% Test

```
splitter_val = GroupShuffleSplit(n_splits=1, train_size=0.50,
    random_state=SEED)
val_idx, test_idx = next(splitter_val.split(X_temp, y_temp,
    groups_temp))
X_val, y_val = X_temp.iloc[val_idx], y_temp.iloc[val_idx]
X_test, y_test = X_temp.iloc[test_idx], y_temp.iloc[test_idx]
```

Implementation of splitting dataset into validation and test sets

By creating this isolated training environment, we ensure that all subsequent calculations such as computing the correlation for feature selection are based exclusively on the training engines, eliminating data leakage.

Correlation analysis between the remaining sensors and RUL

To identify the most informative sensors, we performed a correlation analysis between each sensor and the RUL. The correlation was calculated using `pandas.DataFrame.corr()`, which by default computes the Pearson correlation coefficient.

Columns with higher absolute correlation are more strongly related to RUL and are more informative for modeling, while columns with low correlation can be considered weakly related and may be candidates for exclusion. This threshold was used as a visual reference in our analysis.

```
sensor_cols_cand = [c for c in df_train_raw.columns if "sensor
    " in c or "setting" in c]
corr_rul = (
    df_train_raw[sensor_cols_cand + ["RUL"]]
    .corr()["RUL"]
    .drop("RUL")
    .abs()
    .rename("corr_rul")
)
score_df = pd.DataFrame({"corr_rul": corr_rul})
```

Implementation of correlation analysis calculation

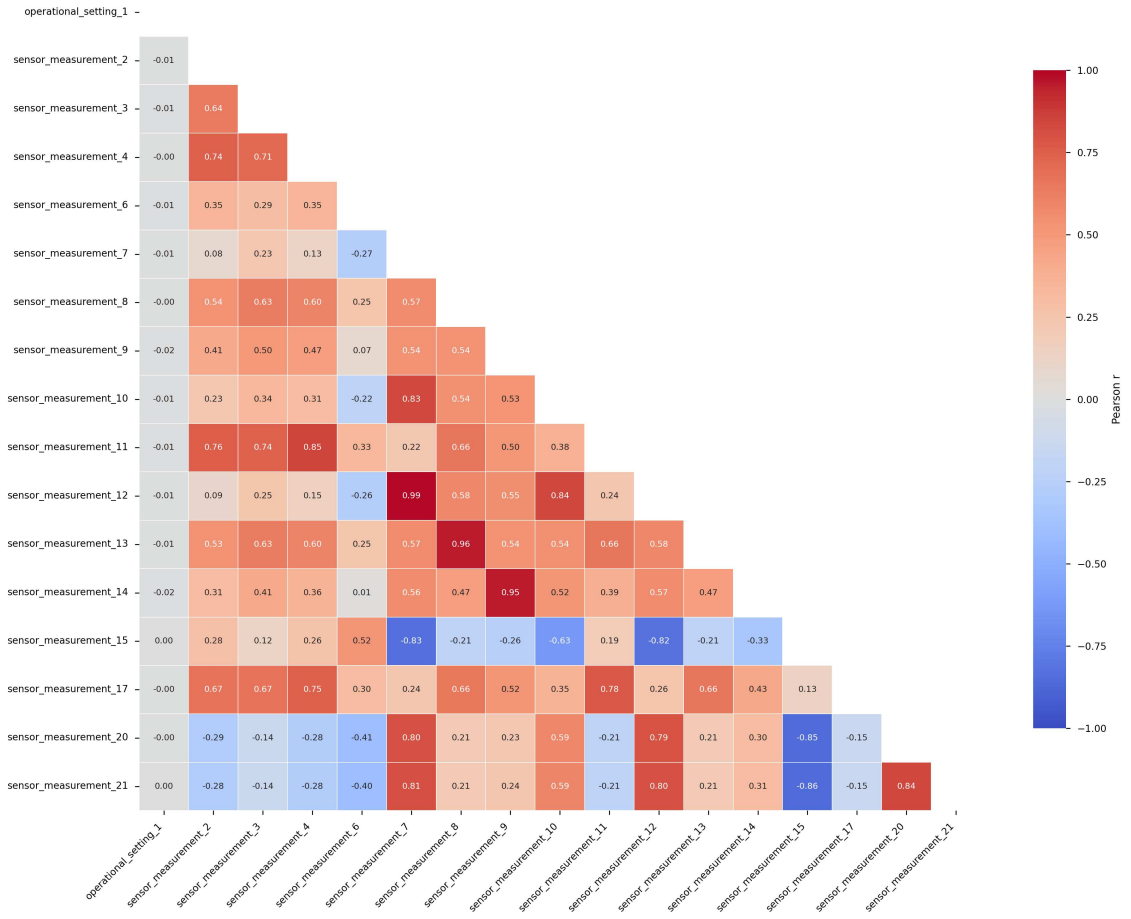


Figure 5.4: Correlation matrix

This analysis allows us to prioritize sensors for feature selection in predictive modeling. Sensors with high absolute correlation with RUL are likely to capture degradation trends effectively, while sensors with very low correlation provide little information and can be ignored in modeling steps.

Here we select features that fit into predefined threshold.

```
target_sensors = score_df[
    (score_df["correlation_rul"] >= SELECTION_THRESHOLD) &
    (score_df.index.str.contains("sensor"))
].index.tolist()
```

Defining target sensors based on correlation threshold

5.3.1 Feature Engineering

Based on prior correlation analysis, we isolate highly informative features. Now we can implement feature engineering widely discussed in Section 3.1.3

Developing new features via catch22

We define a rolling window of size $W = 30$ cycles. As we mentioned in methodology, we should escape data leakage where the end of one engine's cycle incorrectly influences the beginning of another's, and we group the dataset by `unit_number` prior to applying the rolling window. Because rolling window calculations inherently require historical data, the first $W - 1$ cycles for every engine result in undefined values.

```
catch22_names = pycatch22.catch22_all(np.arange(WINDOW_SIZE,
    dtype=float))["names"]

def extract_catch22_features(series: pd.Series) -> pd.
    DataFrame:
     arr = series.to_numpy(dtype=float)
     if len(arr) < WINDOW_SIZE:
         return pd.DataFrame(np.nan, index=series.index,
            columns=catch22_names)
     windows = sliding_window_view(arr, WINDOW_SIZE)
     feats = [pycatch22.catch22_all(w)["values"] for w in
        windows]
     pad = np.full((WINDOW_SIZE - 1, len(catch22_names)), np.
        nan)
     return pd.DataFrame(
         np.vstack([pad, feats]),
         index=series.index,
         columns=catch22_names,
     )
```

Implementation of the `catch22` Feature Extraction Function

The `sliding_window_view` from NumPy creates a sequence of 30-cycle overlapping arrays. The problem in the first attempt is the data leakage. The extracted features must align strictly with the last cycle of their respective window. We address the problem by vertically stacking an array of NaNs at the beginning of the trajectory using `np.full`.

We define function `apply_feature_engineering` where apply `extract_catch22_features` iteratively across the selected sensors in Section 5.3:

```
def apply_feature_engineering(df_subset, target_sensors,
    train_medians=None):
     feature_df = df_subset.copy().reset_index(drop=True)
     catch22_list = []

     for sensor in target_sensors:
         c22_feats = feature_df.groupby("unit_number",
            group_keys=False)[sensor].apply(
             extract_catch22_features
         )
         c22_feats.columns = [f"{sensor}_{col}" for col in
            c22_feats.columns]
         catch22_list.append(c22_feats)

     feature_df = pd.concat([feature_df] + catch22_list, axis
        =1)
```

Implementation of the `apply_feature_engineering` function, utilizing a vectorized approach for applying `extract_catch22_features`.

Following the extraction process, it was observed that some catch22 properties become undefined. This happens when a sensor exhibits zero variance within a specific 30-cycle window, so catch22 cannot calculate its properties [25]. Applying `apply_feature_engineering` on each subset and calculating the sum of all NaNs in the dataframe, we get 23795 missing values in the train set, 3321 in the validation set, 7287 in the test set. In machine learning, missing values in the raw datasets are a very common problem. Various approaches exist to address this:

- Row Removal: A simple approach, involving removing any row that contains a NaN value. We can miss important information for the model using this approach, so it is not a good option [46].
- Imputation: This technique preserves the dataset dimensions by replacing missing values with a calculated statistical estimate derived from the available data. Common metrics include the mean, median, or mode [46].
- Forward and Backward Fill: These methods fill missing values using the closest available values in the column. This method particularly useful for ordered or time-series data [46].

Forward and Backward Fill seems a relevant option for our case, but it can not work if the entire sequence of values is NaN (there are no available close values), so the Imputation method using, for instance, the median, is also a possible approach for this problem. An important part of imputation using the mean is that we should calculate the mean only on the training set, and then reproduce the imputation using this training mean. By doing this, we prevent data leakage.

After testing the count of missing values after Forward and Backward Fill, it reveals that the method successfully fills all missing values. The function for this method:

```
def fill_missing_values(feature_df):
    return feature_df.groupby("unit_number").ffill().bfill()
```

Implementation of function for Forward and Backward Fill of missing values

Then the final version of `apply_feature_engineering` function looks like this:

```
def apply_feature_engineering(df_subset, target_sensors):
    feature_df = df_subset.copy().reset_index(drop=True)
    catch22_list = []
    for sensor in target_sensors:
        c22_feats = feature_df.groupby("unit_number",
           group_keys=False)[sensor].apply(
            extract_catch22_features
        )
        c22_feats.columns = [f"{sensor}_{col}" for col in
           c22_feats.columns]
        catch22_list.append(c22_feats)
    feature_df = pd.concat([feature_df] + catch22_list, axis
       =1)

    filled_feature_df = fill_missing_values(feature_df)

    return filled_feature_df
```

Implementation of the `apply_feature_engineering` function, utilizing a vectorized approach for applying `extract_catch22_features` and handling missing values.

We apply `apply_feature_engineering` on each defined subset, which returns new engineered sets that we will utilize in the next steps:

```
X_train_engineered = apply_feature_engineering(X_train,
   target_sensors)
X_val_engineered = apply_feature_engineering(X_val,
   target_sensors)
X_test_engineered = apply_feature_engineering(X_test,
   target_sensors)
```

Application of `apply_feature_engineering` to the training, validation, and test dataset splits.

Final dropping

We must prevent the most fundamental form of data leakage, when you leave, basically the answers (the RUL or the Label), inside the data that the Random Forest is looking at.

```
columns= ["label", "unit_number", "time_in_cycles", "max_cycle
    ", "RUL"]
columns_to_drop = [c for c in columns if c in
    X_train_engineered.columns]
X_train_engineered.drop(columns=columns_to_drop, inplace=True)
X_val_engineered.drop(columns=columns_to_drop, inplace=True)
X_test_engineered.drop(columns=columns_to_drop, inplace=True)
```

Dropping operational metadata and label columns from the training, validation, and test sets prior to model training.

5.3.2 Class Imbalance Analysis

As discussed in the methodology, we must address the class imbalance. Because the simulated turbofan engines operate in a normal, healthy state for the vast majority of their lifecycles, the rapid degradation phases occur only at the very end of their operational trajectories.

As illustrated in the class distribution figure below, the Safe class significantly outnumbers the Warning and Critical classes across all data splits:

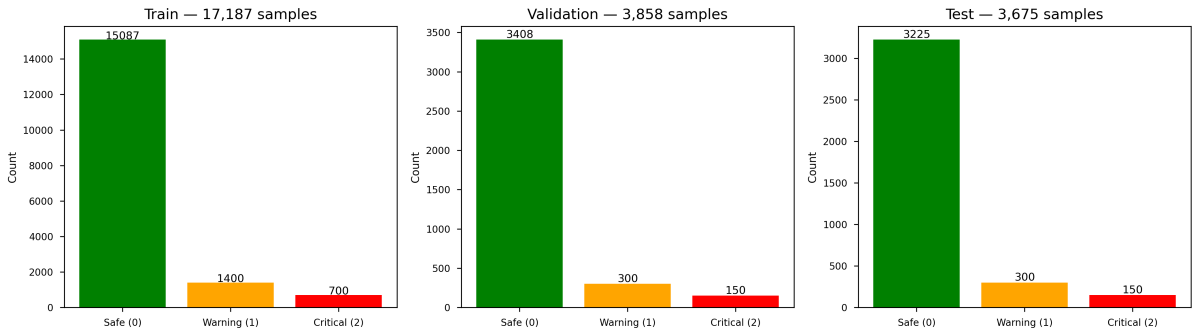


Figure 5.5: Class distribution

5.4 Training

Baseline model training

First, we attempt to train vanilla `RandomForestClassifier` [47] and look at the results. The training pipeline is threefold.

We initialize the classification mode and define a constant parameter `SEED = 42`, which is passed to the algorithm's random state. It is used for internal stochastic calculation. By default, it is a random value, but we set it to a constant so as not to get different results during repeated executions of the pipeline. Next, the model is trained on the training set, which we developed in Section 5.3.1. Finally, we perform testing on the test set to see how the model performs.

```
rf = RandomForestClassifier(random_state=SEED)
rf.fit(X_train_engineered, y_train)
y_pred = rf.predict(X_test_engineered)
```

Initialization and training of the Baseline Random Forest classifier to generate predictions for the test set.

Table 5.1: Classification Metrics

Class Code	Description	Precision	Recall	F1-Score	Support
0	Safe	0.98	0.99	0.99	3225
1	Warning	0.79	0.77	0.78	300
2	Critical	0.94	0.73	0.82	150
Accuracy				0.96	3675
Macro Avg		0.90	0.83	0.86	3675
Weighted Avg		0.96	0.96	0.96	3675

Results show that the overall accuracy is about 0.96, which looks satisfying, but as we discussed in Section 3.2, we should consider Recall and Precision as primary metrics for evaluation. The baseline model performs very well on classifying the Safe class. Looking at the confusion matrix in Figure 5.4, we can clearly see that it classifies almost all Safe classes (3205/3225), but there is a drop in performance in classifying the warning and critical minorities. The confusion matrix proves what we discussed in 4.1, the model struggles to separate neighboring classes.

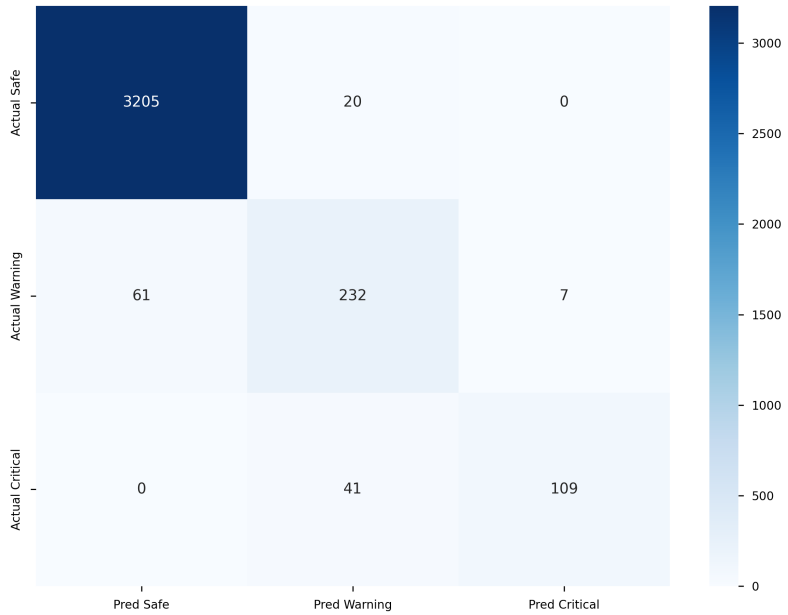


Figure 5.6: Confusion matrix of baseline model

Feature Importance

One of the primary advantages of utilizing a `RandomForestClassifier` is its inherent interpretability. During the training phase, the algorithm automatically calculates the importance of each feature by measuring the Mean Decrease in Impurity, often referred to as Gini importance. This metric quantifies exactly how much each individual feature reduces uncertainty when the decision trees classify the engine's health state.

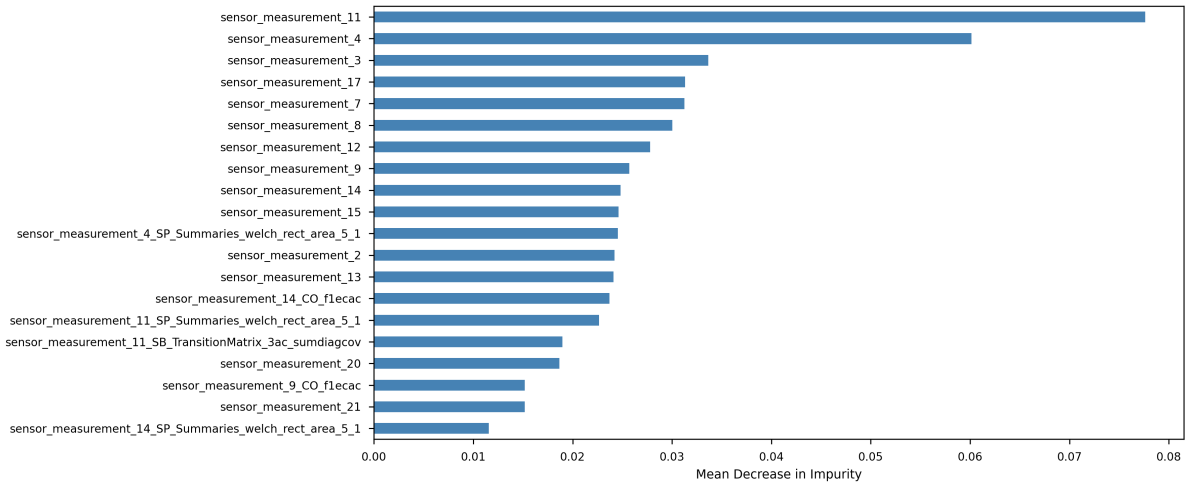


Figure 5.7: 20 the most important features by mean Decrease in Impurity

Following this analysis, it became clear that some of the engineered features did not contribute much to the model's decision. Retaining these features introduces computational complexity, and the model can overfit. As we have a large number of features due to previous feature engineering, we suggest defining a threshold (0.001) for dropping lowly important features.

```
low_importance_columns = feature_importances[
    feature_importances < 0.001].index

X_train_engineered = X_train_engineered.drop(columns=
    low_importance_columns)
X_val_engineered = X_val_engineered.drop(columns=
    low_importance_columns)
X_test_engineered = X_test_engineered.drop(columns=
    low_importance_columns)
```

Dropping redundant features from subsets

As a result of this, the feature space is optimized. The training set was reduced from 237 initial features down to 116 features.

5.4.1 Cross-Validation with Hyperparameter Tuning

Next, we set up a grid search to test different combinations of hyperparameters. The `GridSearchCV` will search all combinations of hyperparameters within the predefined grid. The predefined grid consists of a few parameters, discussed in Section 4.3: the total number of trees (`n_estimators`), how deep those trees are allowed to grow (`max_depth`), and the minimum number of samples left at the end of a branch (`min_samples_leaf`). Also, the search space has a list of class weight parameters. We suggest trying our defined class weights over the basic "balanced" mode, where it automatically adjusts weights inversely proportional to class frequencies in the input data [47].

Formally defined as:

$$w_i = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_i} \tag{5.3}$$

```
param_grid_smote = {
    "rf__n_estimators":     [200, 300],
    "rf__max_depth":        [20, 30],
    "rf__min_samples_leaf": [3, 5],
    "rf__class_weight": [
        "balanced",
        {0: 3.0, 1: 1.0, 2: 3.0},
    ]
}
```

Defining the Hyperparameter Search Space

To focus on Precision of warning class and properly evaluate these combination, there is an option of defining our own score that will be used during grid search. The scoring metric defined using `average='macro'` and `labels=[1]`. The scorer calculates the Precision for every class it sees in a list, which is just Class 1, and then calculates mean. This configuration isolates the precision of the warning class.

```
precision_scorer = make_scorer(precision_score, labels=[1],
    average='macro')
```

Defining own scorer for cross-validation process

As we discussed in section 4.7, we establish k-fold cross-validation with grouping, grouping done by the id of the engine (`unit_number`). The "k" in k-fold means the number of folds. The smaller number of folds can lead to a more biased model. The larger the number increase variance. In practice, the parameter is often set to 5 or 10, in our case, we set it to 5, which should be enough.

```
kf = GroupKFold(n_splits=5)
groups = X_train["unit_number"]
```

Optimizing the classifier using Grouped Grid Search

To implement the synthetic oversampling, we configure the SMOTE algorithm. Standard practice suggests that minority classes are oversampled in proportion 1:1 of the majority class. In our case, the Safe class is huge, so the proportion 1:1 might be overwhelming. After testing different proportions, the `target_minority_count` of 5000 chosen as the final best performing strategy from the context of time complexity and performance results.

```
target_minority_count = 5000
smote = SMOTE(
    sampling_strategy={1: target_minority_count, 2:
       target_minority_count},
    random_state=SEED,
)
rfc = RandomForestClassifier(random_state=SEED, n_jobs=-1)

pipeline = ImbPipeline([
    ('smote', smote),
    ('rf', rfc)
])
```

Synthetic Oversampling using SMOTE

Crucially, integrating SMOTE into a cross-validation introduces a severe risk of data leakage. To prevent this, the `Pipeline` from the `imbalanced-learn` library is utilized. Unlike a standard Scikit-Learn pipeline, the `ImbPipeline` guarantees that during cross-validation, the SMOTE transformation is applied exclusively to the training subset of each fold.

```
rfc = RandomForestClassifier(random_state=SEED, n_jobs=-1)
pipeline = ImbPipeline([
    ('smote', smote),
    ('rf', rfc)
])
```

Pipeline definition

The final stage of the tuning methodology involves the execution of a global hyperparameter search utilizing the SMOTE-Pipeline.

```
grid_search_smote = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid_smote,
    cv=kf,
    scoring=precision_scorer,
    n_jobs=1,
    verbose=1
)

grid_search_smote.fit(X_train_engineered, y_train, groups=
    groups)
```

Defining 5-Fold Cross-Validation with Hyperparameter Tuning using Grid Search

During this process, the `GridSearchCV` orchestrates the 5-fold `GroupKFold` cross-validation. In each iteration, the dataset is partitioned in a way that four engine groups are used for training and one is used for validation. Within the training partition of each fold, the `ImbPipeline` invokes SMOTE to synthetically expand the minority classes. The `RandomForestClassifier` is then trained on this balanced subset and evaluated using the `precision_scorer`.

Table 5.2: Best Parameters Results

Metric / Parameter	**Optimized Value**
Hyperparameters	
`n_estimators`	300
`max_depth`	30
`min_samples_leaf`	5
`class_weight`	{0: 3.0, 1: 1.0, 2: 3.0}
Performance Scores	
Mean Cross-Validation Score	0.7618
Test Set Precision (Warning class)	0.8056

The model achieved a mean cross-validation precision of 0.7618 for the Warning class. Notably, the final precision on the test set reached 0.8056. Usually, we expect performance to drop slightly on unseen data, so the fact that the test score is higher suggests that the model generalizes well.

5.4.2 Final Evaluation on Test Set

After determining the best hyperparameters through the SMOTE-integrated grid search, the final model was evaluated on the engine-isolated test set. The objective is to compare this optimized configuration against the baseline Random Forest results presented in Table 5.3. From the predictive maintenance perspective, the most critical improvement must be found in the Recall of the Warning and Critical classes, as the baseline model failed to identify a significant portion of degrading engines.

Table 5.3: Baseline Random Forest Performance Metrics

Class Code	Description	Precision	Recall	F1-Score	Support
0	Safe	0.98	0.99	0.99	3225
1	Warning	0.79	0.77	0.78	300
2	Critical	0.94	0.73	0.82	150
Accuracy				0.96	3675
Macro Avg		0.90	0.83	0.86	3675
Weighted Avg		0.96	0.96	0.96	3675

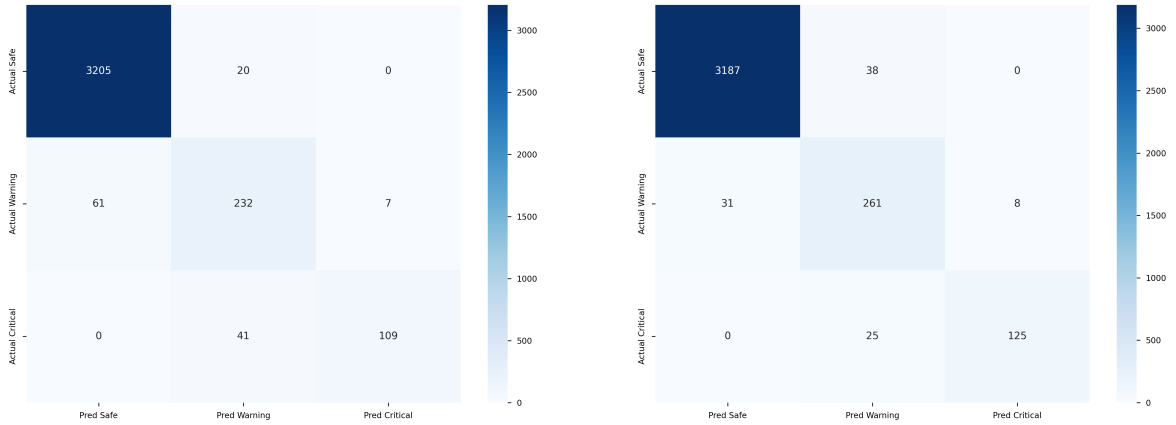
The final tuned model performance is presented in Table 5.4. The results show a clear enhancement across all minority class metrics. We can separate the improvements into two key areas: Recall improvement and Precision stability.

Table 5.4: Final Tuned Model Performance Metrics

Class Code	Description	Precision	Recall	F1-Score	Support
0	Safe	0.99	0.99	0.99	3225
1	Warning	0.81	0.87	0.84	300
2	Critical	0.94	0.83	0.88	150
Accuracy				0.97	3675
Macro Avg		0.91	0.90	0.90	3675
Weighted Avg		0.97	0.97	0.97	3675

The baseline model struggled with a Recall of 0.77 for the Warning class and 0.73 for the Critical class. In contrast, the tuned model achieved 0.87 and 0.83, respectively. This means that the model is much more capable of catching an engine that needs maintenance before it reaches a critical point. Furthermore, by looking at the confusion matrices in Figure 5.8, the false negative instances where actual Warning or Critical classes were predicted as Safe were reduced by nearly half.

As we discussed earlier, Recall and Precision are dependent variables, so while increasing Recall, usually, Precision drops, but due to our custom class weight, the warning precision actually increased from 0.79 to 0.81.



(a) Baseline Random Forest (b) Tuned Random Forest

Figure 5.8: Comparison of Confusion Matrices

Model Interpretability

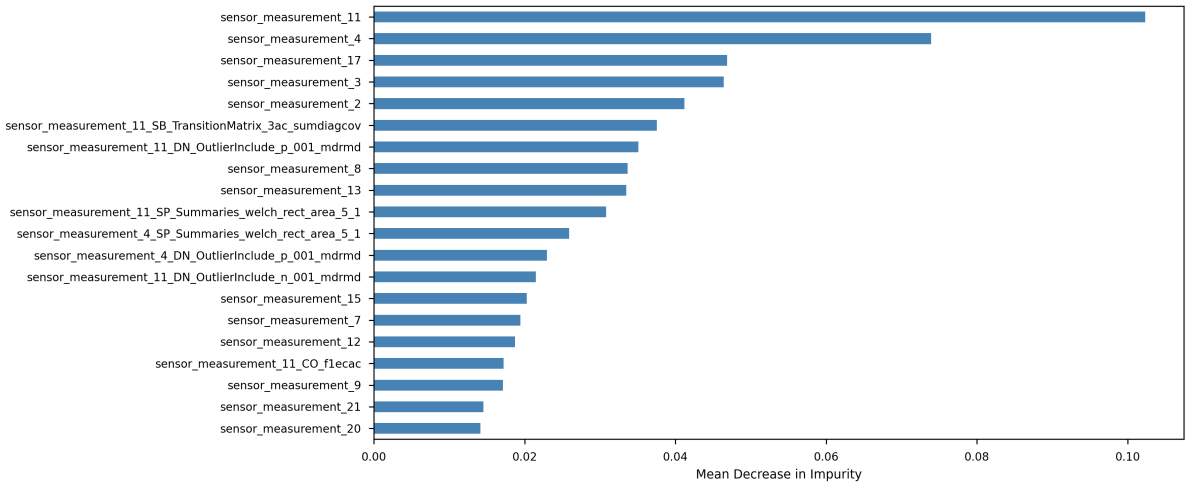


Figure 5.9: Top 20 most important features ranked by Mean Decrease in Impurity

From a C-MAPSS perspective, the high ranking of specific sensors validates the model's logic against known thermodynamic trends. As we know, engines in the FD003 dataset experience HPC Degradation or Fan Degradation, so the feature selection process must highlight parameters that respond to this fact. As shown in Figure 5.9, the most influential features are derived from Sensor 11 (Ps30 - Static pressure at HPC outlet, according to Table 3.5), Sensor 4 (T50 - Total temperature at LPT outlet), and Sensor 17 (htBleed - Bleed enthalpy). This interpretability transforms the model from a black box into a transparent tool.

Chapter 6

Discussion

The primary objective of this work was to shift the initial continuous prediction of the Remaining Useful Life of turbofan engines using NASA C-MAPSS data into a discrete, multi-class classification problem.

6.1 Difficulties Experienced

During our work, the most frequent problem and complexity seems to be handling the time-series nature of data, where an improper approach leads to data leakage. Expanded time-series data presents another major obstacle. Long training time, which occurs due to generating synthetic samples while simultaneously evaluating different hyperparameter combinations during grid search, creates a computational challenge.

6.2 Achieved Results Compare to Other Works

It is necessary to compare our final results against the existing shifts continuous prediction of the Remaining Useful Life into a discrete, multi-class classification problem, discussed in Section 4.1. Basically, the challenge of previous works was achieving high performance across the transitional states without suffering from significant misclassification between neighboring classes.

Comparing overall global metrics, our tuned Random Forest achieved a test accuracy of 97%, which significantly outperforms the 70% accuracy reported by Chaibi et al. [31] using CatBoost and the 81% accuracy achieved by Soni et al. [34] using a standard Random Forest. As previously mentioned, global accuracy is not the right metric for our problem. The true improvement of our methodology is demonstrated within the minority class metrics.

In the work by Chaibi et al. [31], the CatBoost model was found to be the most balanced among their tested algorithms, but it struggled with the neighboring classes. For the moderate degradation state (equivalent to our Warning class, which we identify as a window for maintenance), they reported a Precision of 0.39 and a Recall of 0.63. Furthermore, their Alert state (Critical) achieved a Recall of only 0.52. Our model achieved a Warning Precision of 0.81 and a Recall of 0.87. Additionally, our Critical class Recall reached 0.83, outperforming the 0.52 recall of the CatBoost model in [31].

Similarly, with Soni et al. [34]. Here it is even more relevant because their approach utilizes Random Forest, achieving Precision fo Warning class equal to our 0.81, but we

outperform in Recall 0.87, compared to their 0.70. Additionally, we outperform the classification of Critical class, with Precision 0.94 and Recall 0.83, compared to their 0.87 and 0.82.

Therefore, the analysis confirms that the methodology proposed in this work provides a comparable solution.

6.3 Practical Applicability of the Results

As we discussed in Section 2.2, providing exact continuous RUL can create ambiguity for ground crews. Our proposed discrete outputs of the model map to specific operational actions: Normal Operation (Safe), Window for maintenance (Warning), or Immediate action (Critical).

As demonstrated in the feature importance analysis, the model's predictions are driven by physically relevant sensors, such as Sensor 11 (Static pressure at HPC outlet) and Sensor 4 (Total temperature at LPT outlet). For a maintenance planner, this can be an example of a transparent decision-support tool.

6.4 Limitations of the NASA C-MAPSS Data

It is necessary to discuss the limitations of the data. The NASA C-MAPSS dataset, despite being an industry standard for predictive maintenance research, is simulated. As Saxena et al. [2] mentioned, there is a lack of open data from aircraft companies, and they had to assume normal noise distributions based on existing literature. In the dataset, degradation was limited to the High-Pressure Compressor (HPC) module [2], so data did not include the degradation of other engine modules.

6.5 Recommendations for Future Development

The developed pipeline, specifically the combination of `catch22` feature extraction, threshold-based feature selection, and SMOTE integrated cost-sensitive learning, should be tested on the more complex C-MAPSS datasets, such as FD004. Because FD004 includes six different operational settings.

Chapter 7

Conclusion

This work was dedicated to the PdM field, where we developed a machine learning model to classify turbofan engine health states. All goals and partial objectives have been successfully completed.

We made logical thresholds based on typical industrial maintenance practice to categorize engine health into Safe, Warning, and Critical classes.

The implemented machine learning pipeline, which integrated dimensionality reduction, catch22 feature engineering, and the Random Forest algorithm, achieved robust results. The final tuned model achieved a global accuracy of 0.97, with a high macro-average Precision of 0.91 and a high macro-average Recall of 0.90. This precision performance confirms that the developed solution effectively minimizes false alarms.

The requirement for model interpretability was met through the Random Forest's ability to perform feature importance analysis. This allowed for the identification of key parameters that are highly correlated with engine health.

In conclusion, the methodology proposed and implemented in this work offers an alternative to traditional regression methods.

Bibliography

1. GOOGLE. *Gemini 3 Flash*. 2026. Gemini 3. Available also from: https://gemini.google.com/. Large Language Model.
2. SAXENA, A.; GOEBEL, K.; SIMON, D.; EKLUND, N. Damage propagation modeling for aircraft engine run-to-failure simulation. In: *2008 International Conference on Prognostics and Health Management*. 2008, pp. 1–9. Available from DOI: 10.1109/PHM.2008.4711414.
3. POOR, P.; ŽENÍŠEK, D.; BASL, J. Historical Overview of Maintenance Management Strategies: Development from Breakdown Maintenance to Predictive Maintenance in Accordance with Four Industrial Revolutions. 2019.
4. BARU, A.; JOHNSON, R. *Three Ways to Estimate Remaining Useful Life for Predictive Maintenance* [MathWorks]. 2023. Available also from: https://www.mathworks.com/company/technical-articles/three-ways-to-estimate-remaining-useful-life-for-predictive-maintenance.html.
5. HE, W.; BAIG, M. J. A.; IQBAL, M. T. An Internet of Things—Supervisory Control and Data Acquisition (IoT-SCADA) Architecture for Photovoltaic System Monitoring, Control, and Inspection in Real Time. *Electronics*. 2025, vol. 14, no. 1. ISSN 2079-9292. Available from DOI: 10.3390/electronics14010042.
6. TEIXEIRA, H. N.; LOPES, I.; BRAGA, A. C. Condition-based maintenance implementation: a literature review. *Procedia Manufacturing*. 2020, vol. 51, pp. 228–235. ISSN 2351-9789. Available from DOI: https://doi.org/10.1016/j.promfg.2020.10.033. 30th International Conference on Flexible Automation and Intelligent Manufacturing (FAIM2021).
7. PATIBANDLA, K. R. Predictive Maintenance in Aviation using Artificial Intelligence. *Journal of Artificial Intelligence General science (JAIGS) ISSN:3006-4023*. 2024, vol. 4, pp. 325–333. Available from DOI: 10.60087/jaigs.v4i1.214.
8. ROLLS-ROYCE. *IntelligentEngine: how AI scales up IoT capability in turbofan jet engines*. 2020. Available also from: https://www.rolls-royce.com/media/our-stories/discover/2020/intelligentengine-how-ai-scales-up-iot-capability-in-turbofan-jet-engines.aspx.
9. AL-DULAIMI, A.; ZABIHI, S.; ASIF, A.; MOHAMMADI, A. A multimodal and hybrid deep neural network model for Remaining Useful Life estimation. *Computers in Industry*. 2019, vol. 108, pp. 186–196. ISSN 0166-3615. Available from DOI: https://doi.org/10.1016/j.compind.2019.02.004.
10. HEIMES, F. Recurrent neural networks for remaining useful life estimation. In: 2008, pp. 1–6. Available from DOI: 10.1109/PHM.2008.4711422.

11. WU, Y.; YUAN, M.; DONG, S.; LIN, L.; LIU, Y. Remaining Useful Life Estimation of Engineered Systems using vanilla LSTM Neural Networks. *Neurocomputing*. 2018, vol. 275, pp. 167–179. Available from DOI: 10.1016/j.neucom.2017.05.063.

12. VOLLERT, S.; THEISSLER, A. Challenges of machine learning-based RUL prognosis: A review on NASA's C-MAPSS data set. In: *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2021, pp. 1–8. Available from DOI: 10.1109/ETFA45728.2021.9613682.

13. AMERICAN HONDA MOTOR CO., INC. *Honda Maintenance Minder System* [[online]]. 2018. [visited on 2026-01-18]. Available from: https://owners.honda.com/utility/download?path=/static/pdfs/2018/CR-V/2018_CR-V_Maintenance_Minder_System.pdf.

14. MAHESH, B. Machine Learning Algorithms -A Review. *International Journal of Science and Research (IJSR)*. 2019, vol. 9. Available from DOI: 10.21275/ART20203995.

15. RUSSELL, S. J.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4th. Hoboken, NJ: Pearson, 2020.

16. SCIKIT-LEARN DEVELOPERS. *Decision Trees* [https://scikit-learn.org/stable/modules/tree.html]. 2024.

17. JIM. *How to Avoid Overfitting?* 2022. Available also from: https://www.rbloggers.com/2022/09/how-to-avoid-overfitting/.

18. BOLDMETHOD. *How A Turbofan Engine Works: The Basic Steps*. 2015. Available also from: https://www.boldmethod.com/learn-to-fly/aircraft-systems/how-does-a-jet-engine-turbofan-system-work-the-basic-steps/.

19. MATHWORKS. *MATLAB version 9.14.0 (R2023a)*. Natick, Massachusetts, United States: The MathWorks Inc., 2023.

20. MATHWORKS. *Simulink version 10.7 (R2023a)*. Natick, Massachusetts, United States: The MathWorks Inc., 2023.

21. HONG, C. W.; LEE, C.; LEE, K.; KO, M.-S.; KIM, D.; HUR, K. Remaining Useful Life Prognosis for Turbofan Engine Using Explainable Deep Neural Networks with Dimensionality Reduction. *Sensors*. 2020, vol. 20, pp. 1–19. Available from DOI: 10.3390/s20226626.

22. SOOFI, A.; AWAN, A. Classification Techniques in Machine Learning: Applications and Issues. *Journal of Basic Applied Sciences*. 2017, vol. 13, pp. 459–465. Available from DOI: 10.6000/1927-5129.2017.13.76.

23. FAOUZI, J. Time Series Classification: A Review of Algorithms and Implementations. In: 2024. ISBN 978-0-85466-053-7. Available from DOI: 10.5772/intechopen.1004810.

24. IRANI, H.; GHAHREMANI, Y.; KERMANI, A.; METSIS, V. Time Series Embedding Methods for Classification Tasks: A Review. *Expert Systems*. 2025, vol. 42. Available from DOI: 10.1111/exsy.70148.

25. CATCH22 DEVELOPERS. *catch22 Official Documentation*. 2026. Available also from: https://time-series-features.gitbook.io/catch22.

26. TSFRESH DEVELOPERS. *tsfresh Official Documentation*. 2026. Available also from: https://tsfresh.readthedocs.io/en/latest/.

27. CONGALTON, R.; GREEN, K. *Assessing the Accuracy of Remotely Sensed Data: Principles and Practices, Third Edition*. 2019. ISBN 9780429052729. Available from DOI: 10.1201/9780429052729.

28. FARHADPOUR, S.; WARNER, T.; MAXWELL, A. Selecting and Interpreting Multiclass Loss and Accuracy Assessment Metrics for Classifications with Class Imbalance: Guidance and Best Practices. *Remote Sensing*. 2024, vol. 16, p. 533. Available from DOI: 10.3390/rs16030533.

29. VAKILI, M.; GHAMSARI, M.; REZAEI, M. Performance Analysis and Comparison of Machine and Deep Learning Algorithms for IoT Data Classification. 2020. Available from DOI: 10.48550/arXiv.2001.09636.

30. SALMAN, H.; KALAKECH, A.; STEITI, A. Random Forest Algorithm Overview. *Babylonian Journal of Machine Learning*. 2024, vol. 2024, pp. 69–79. Available from DOI: 10.58496/BJML/2024/007.

31. CHAIBI, A.; STITOU, A.; LABIBA, B.; TCHOFFA, D.; SERROU, D.; EL MHAMEDI, A.; LAGRAT, I. Classification of Engine Health States: Approaches to Predict Degradation. *IFAC-PapersOnLine*. 2025, vol. 59, no. 10, pp. 1856–1861. ISSN 2405-8963. Available from DOI: https://doi.org/10.1016/j.ifacol.2025.09.312.

32. CATBOOST DEVELOPMENT TEAM. *CatBoost Documentation*. 2024. Available also from: https://catboost.ai/docs/en/.

33. XGBOOST DEVELOPERS. *XGBoost Documentation (Release 3.2.0)*. 2024. Available also from: https://xgboost.readthedocs.io/en/release_3.2.0/.

34. SONI, P.; ANAS KHAN Mohd adnd Zubair, M.; KUMAR GARG, S. Multiclass classification for predicting Remaining Useful Life (RUL) of the turbofan engine. In: *2021 11th International Conference on Cloud Computing, Data Science Engineering (Confluence)*. 2021, pp. 1023–1028. Available from DOI: 10.1109/Confluence51648.2021.9377131.

35. YILDIRIM, U.; AFŞER, H. Linear Methods for Predictive Maintenance: The Case of NASA C-MAPSS Datasets. *Applied Sciences*. 2025, vol. 15, p. 9945. Available from DOI: 10.3390/app15189945.

36. SCHOBER, P.; BOER, C.; SCHWARTE, L. A. Correlation Coefficients: Appropriate Use and Interpretation. 2018, vol. 126, no. 5, pp. 1763–1768. Available from DOI: 10.1213/ANE.0000000000002864.

37. STATISTICS SOLUTIONS. *Pearson's Correlation Coefficient*. 2024. Available also from: https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/pearsons-correlation-coefficient/.

38. WAHID, A.; BRESLIN, J.; ALI, M. I. TCRSCANet: Harnessing Temporal Convolutions and Recurrent Skip Component for Enhanced RUL Estimation in Mechanical Systems. *Human-Centric Intelligent Systems*. 2024, vol. 4, pp. 1–24. Available from DOI: 10.1007/s44230-023-00060-0.

39. LUMUMBA, V.; SANG, D.; MPAINE, M.; NJOKA, G.; MUSYIMI, D.; KAVITA. Comparative Analysis of Cross-Validation Techniques: LOOCV, K-folds Cross-Validation, and Repeated K-folds Cross-Validation in Machine Learning Models. *American Journal of Theoretical and Applied Statistics*. 2024, vol. 13, pp. 127–137. Available from DOI: 10.11648/j.ajtas.20241305.13.

40. BOTACHE, D.; DINGEL, K.; HUHNSTOCK, R.; EHRESMANN, A.; SICK, B. *Unraveling the Complexity of Splitting Sequential Data: Tackling Challenges in Video and Time Series Analysis*. 2023. Available from arXiv: 2307.14294 [cs.LG].

41. CHAWLA, N.; BOWYER, K.; HALL, L.; KEGELMEYER, W. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*. 2002, vol. 16, pp. 321–357. Available from DOI: 10.1613/jair.953.

42. CHEN, C.; LIAW, A.; BREIMAN, L. *Using random forest to learn imbalanced data*. Berkeley, CA, 2004. Tech. rep., 666. University of California, Berkeley. Available also from: https://statistics.berkeley.edu/sites/default/files/tech-reports/666.pdf.

43. PANDAS CONTRIBUTORS. *Pandas Documentation*. Pandas, 2026. Available also from: https://pandas.pydata.org/docs/getting_started/overview.html.

44. NUMPY CONTRIBUTORS. *NumPy Documentation*. NumPy, 2026. Available also from: https://numpy.org/doc/2.4/user/whatisnumpy.html.

45. SCIKIT-LEARN CONTRIBUTORS. *scikit-learn Documentation*. scikit-learn, 2026. Available also from: https://scikit-learn.org/stable/user_guide.html#.

46. GEEKSFORGEEKS. *Handling Missing Values in Machine Learning* [https://www.geeksforgeeks.org/data-analysis/handling-missing-values-machine-learning/]. 2025.

47. SCIKIT-LEARN DEVELOPERS. *sklearn.ensemble.RandomForestClassifier* [https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html]. 2026.

Appendices

Appendix A

Source Code and Dataset

The Python notebook and dataset are available on the Kaggle platform. The dataset consists of the raw train set and target RUL from the initial C-MPASS FD003 dataset and does not include developed health states labels.

`https://www.kaggle.com/code/artsiomhalachkin/c-mapss-classification`